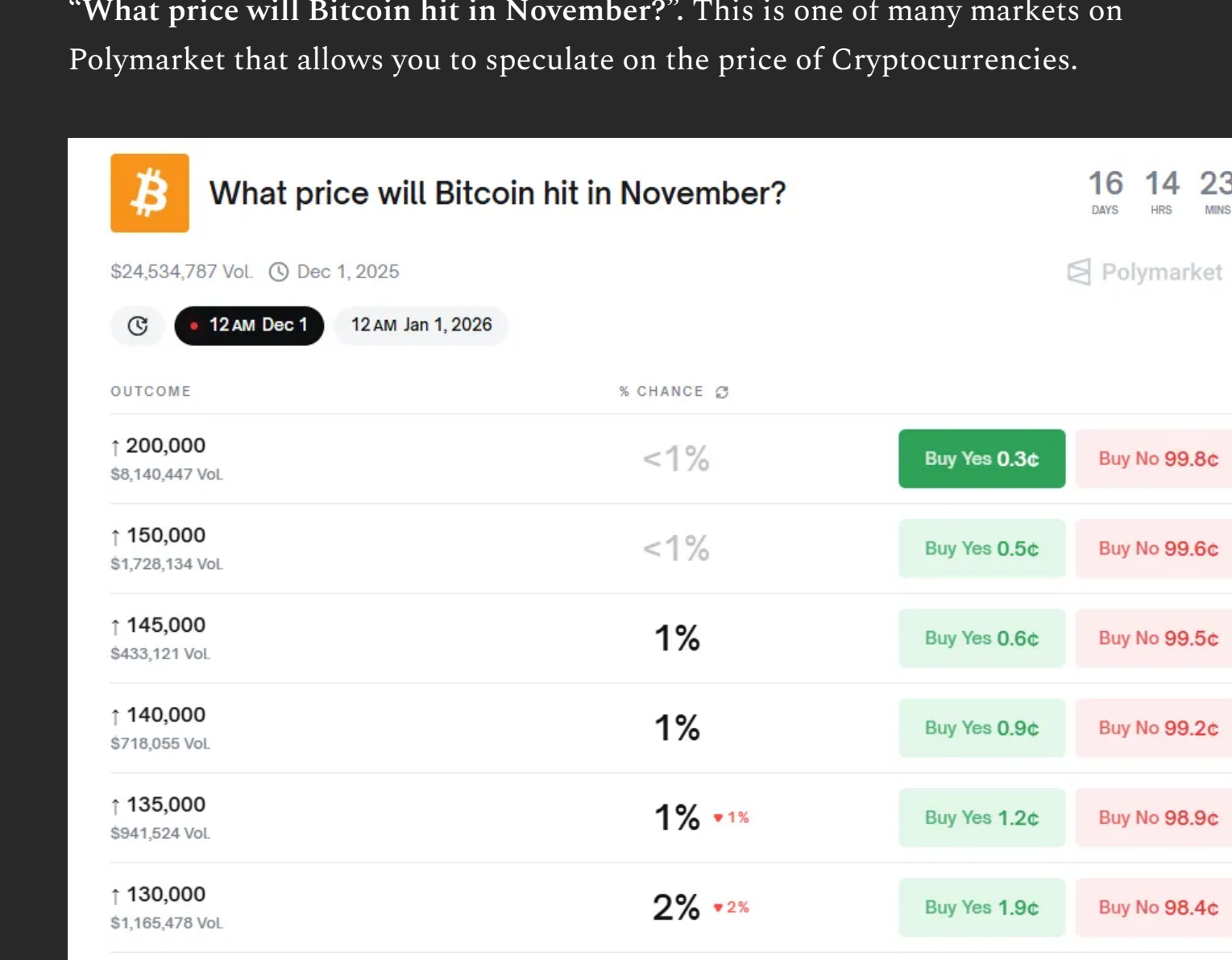
How to Price Touch-Style Options (Polymarket)

And how to profit from mispricings

VERTOX
NOV 16, 2025

1 1 Share

“**What price will Bitcoin hit in November?**”. This is one of many markets on Polymarket that allows you to speculate on the price of Cryptocurrencies.



What price will Bitcoin hit in November?

16 DAYS 14 HRS 23 MINS

$24,534,787 Vol. Dec 1, 2025

Polymarket

12 AM Dec 1 | 12 AM Jan 1, 2026

OUTCOME	% CHANCE		
↑ 200,000 $8,140,447 Vol.	<1%	Buy Yes 0.3¢	Buy No 99.8¢
↑ 150,000 $1,728,134 Vol.	<1%	Buy Yes 0.5¢	Buy No 99.6¢
↑ 145,000 $433,121 Vol.	1%	Buy Yes 0.6¢	Buy No 99.5¢
↑ 140,000 $718,055 Vol.	1%	Buy Yes 0.9¢	Buy No 99.2¢
↑ 135,000 $941,524 Vol.	1% ▾1%	Buy Yes 1.2¢	Buy No 98.9¢
↑ 130,000 $1,165,478 Vol.	2% ▾2%	Buy Yes 1.9¢	Buy No 98.4¢
↑ 125,000 $848,811 Vol.	3% ▾2%	Buy Yes 2.9¢	Buy No 97.4¢
↑ 120,000 $1,124,604 Vol.	4% ▾5%	Buy Yes 4.2¢	Buy No 96.0¢

What you see in this picture are essentially options contracts!
You can buy a share of “Bitcoin price will be above $130.000 at some point in November” for 1.9 Cents, and if this ends up happening, you will receive $1.0, a profit of $0.981. You can also bet that it won’t happen for the price of $98.4. Both Yes and No have their own order book.

What we will do in this article is price those options using data from a larger options market, Deribit.
We do this because, on average, a bigger and more liquid option market will better reflect fair prices. If Polymarket prices are way out of line with what Deribit implies, then that’s an opportunity to profit!

Setting Up the Environment

Before we can get started pricing those options, let us first set up our environment. This means importing libraries and defining important variables:

```
import numpy as np
import pandas as pd
import math
import json
import os
import re
import time
import urllib.error
import urllib.parse
import urllib.request
from typing import Optional, Tuple, Dict, List
from datetime import datetime, timezone

import matplotlib.pyplot as plt
from matplotlib.ticker import FuncFormatter
from matplotlib import cm
from scipy.interpolate import PchipInterpolator, RegularGridInterpolator
from scipy.stats import gaussian_kde, norm
from scipy.optimize import least_squares

print(”Dependencies ready.”)
```

Here are all of the libraries we are gonna be using in this article. If you don’t have any of them, you can install them via

```
pip install *library name*
```

Now our parameters:

```
# User-editable parameters


# --- Core asset / horizon ---
UNDERLYING = “BTC”                    # “BTC” or “ETH”
EXPIRY_ISO = “2026-01-01”             # Target option/barrier horizon
(YYYY-MM-DD)
VALUATION_DT = None                    # None = now (UTC) ; or “YYYY-MM-DD
HH:MM” (local)

# --- Monte Carlo config ---
N_PATHS_HIT = 200_000                 # Hit probability estimation & sweep
envelopes
N_STEPS = None                        # None = ~daily; or set an int
SEED = 42                             # Reproducibility seed used
everywhere
MBTCOD = “bb”                         # ‘bb’ (bridge), ‘gobet’, or
‘bb_gobet’
USE_MID_SIGMA = True                  # Use mid-step sigma in bridge/shift

# --- K sweep ---
SWEEP_POINTS = 301                    # Number of K points across the grid
S_MIN_FACTOR = 0.25                   # Grid min = factor * spot
S_MAX_FACTOR = 3.00                   # Grid max = factor * spot

# --- Gating (Polymarket comparison) ---
ABS_THRESHOLD = 0.03                  # Must exceed both CI margin and
this absolute threshold

# --- Output locations ---
BASE_DIR = os.getcwd()
SNAPSHOT_DIR = os.path.join(BASE_DIR, “data_snapshots”)
SMILES_DIR = os.path.join(BASE_DIR, “smiles_term_structure”)
OUTPUTS_DIR = os.path.join(BASE_DIR, “outputs”)

# Output filenames (lowercase for convenience)
EXP_STR = EXPIRY_ISO.replace(”-”, “”)
SWEEP_CSV = os.path.join(OUTPUTS_DIR, f”
{UNDERLYING.lower()}_probability_sweep.csv”)
META_JSON = os.path.join(OUTPUTS_DIR, f”
{UNDERLYING.lower()}_probability_sweep_meta.json”)
PLOT_PNG = os.path.join(OUTPUTS_DIR, f”
polymarket_vs_model_{UNDERLYING.lower()}_{EXP_STR}.png”)

# Ensure directories
os.makedirs(SNAPSHOT_DIR, exist_ok=True)
os.makedirs(SMILES_DIR, exist_ok=True)
os.makedirs(OUTPUTS_DIR, exist_ok=True)

print(”Parameters set.”)
```

Some of those should be pretty self-explanatory. The other ones, like the “K sweep” parameters, will be popping up in the code in later chapters.

Data Retrieval

We can’t price anything without data. This section of the article will be all about gathering all of the data we need and putting it into a nice format.

Helpers

First, some useful utilities:

```
def yearfrac_365(start: datetime, end: datetime) -> float:
    return max(0.0, (end - start).total_seconds() / (365.25 * 24 *
3600.0))
```

This function tells us how many years are between start and end, which are both datetime objects.

```
def fetch_url_text(url: str, timeout: float = 10.0) -> Optional[str]:
    try:
        with urllib.request.urlopen(url, timeout=timeout) as resp:
            return resp.read().decode(”utf-8”)
    except Exception:
        return None
```

This function makes a request to the given URL and returns the text of the response.

Risk-free Rates

Option prices depend on risk-free rates, because an option’s payoff can be replicated using the underlying asset and a risk-free bond, and this risk-free bond’s price depends on the risk-free rate.

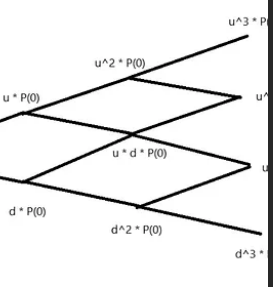
Under risk-neutral valuation, we have:

$$\text{Option Price} = e^{-rT} E[\text{Payoff}]$$

where the expectation is under risk-neutral probabilities.
To learn more about general option pricing like this, check out my dedicated article:

Option Pricing Basics

VERTOX · 26 APR



An options contract gives the buyer the right to buy or sell (call or put) the underlying asset at an agreed upon price and time.

Read full story →

We can grab the latest yearly interest rate directly from FRED.
By default, we will use the **Secured Overnight Financing Rate** (**SOFR**), the most common benchmark for risk-free rates in the US. Our fallback will be the 3-Month Treasury Yield (DGS3MO), the interest on a 3-month US Treasury Bill.

Here is the function to grab those from FRED:

```
def fetch_fred_latest(series_id: str) -> Optional[float]:
    ”””Fetch latest non-missing value from FRED (percent -> decimal).”””
    url = f”https://fred.stlouisfed.org/graph/fredgraph.csv?id=
{series_id}”
    txt = fetch_url_text(url, timeout=10)
    if not txt:
        return None
    lines = [ln.strip() for ln in txt.splitlines() if ln.strip()]
    for ln in reversed(lines[1:]):  # skip header
        parts = ln.split(”,”)
        if len(parts) < 2:
            continue
        val = parts[1].strip()
        if val not in (”.”, “”, “NaN”):
            try:
                return float(val) / 100.0
            except ValueError:
                continue
    return None
```

It’s more convenient to work with continuous interest rates rather than simple annual interest rates.

For the simple annual interest rate R, we have that $1 in T years is worth:

$$\text{Value} = 1 + RT$$

While for a continuous interest rate r, we have that $1 in T years is worth:

$$\text{Value} = e^{rT}$$

If we ask ourselves the opposite question (What is $1 in T years worth today?), we get the so-called “discount factor”, which for the simple linear interest rate is:

$$\text{df} = \frac{1}{1+RT}$$

and for the continuous interest rate:

$$df = e^{-rT}$$

We can equate those two and solve for r to be able to convert linear interest rates to continuous ones:

$$r = \frac{\ln(1+RT)}{T} = -\frac{\ln(\text{df})}{T}$$

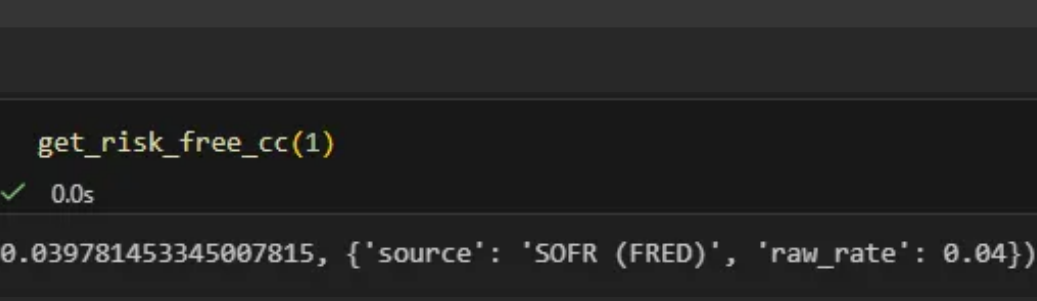
Here is the code that does that:

```
def simple_to_cc(y_simple: float, T_years: float, daycount_base: float,
quote_base: float) -> float:
    ”””
    Convert a simple annualized rate (quoted on quote_base) to
continuous compounding over T_years.
    df = 1 / (1 + y_simple * T_quote) with T_quote = T_years *
(daycount/quote); r_cc = -ln(df) / T_years
    ”””
    T_quote = T_years * (daycount_base / quote_base)
    df = 1.0 / max(1e-12, 1.0 + y_simple * T_quote)
    return -math.log(df) / max(T_years, 1e-12)
```

Note: This code looks a little more complicated than what we just described. The reason for this is that different sources of interest rates use different definitions of what a year is. Some use 360 days, while others use 365 days, etc. This code automatically accounts for that.

Finally, we can combine all those functions into one function that will give us our continuous risk-free rate with a simple function call:

```
def get_risk_free_cc(T: float) -> Tuple[float, Dict[str, float]]:
    ”””
    Compute continuous-comp risk-free over T years.
    Preferred: SOFR (ACT/360) -> CC. Fallback: DGS3MO (approx ACT/365) -
> CC.
    ”””
    meta = {”source”: “”, “raw_rate”: np.nan}
    sofr = fetch_fred_latest(”SOFR”)
    if sofr is not None and sofr > 0.0:
        r = simple_to_cc(y_simple=sofr, T_years=T, daycount_base=365.25,
quote_base=360.0)
        meta.update({”source”: “SOFR (FRED)”, “raw_rate”: sofr})
        return float(r), meta
    tbill3m = fetch_fred_latest(”DGS3MO”)
    if tbill3m is not None and tbill3m > 0.0:
        r = simple_to_cc(y_simple=tbill3m, T_years=T,
daycount_base=365.25, quote_base=365.25)
        meta.update({”source”: “DGS3MO (FRED)”, “raw_rate”: tbill3m})
        return float(r), meta
    meta.update({”source”: “fallback_zero”, “raw_rate”: 0.0})
    return 0.0, meta
```



```
get_risk_free_cc(1)
✓ 0.0s
(0.039781453345007815, {'source': 'SOFR (FRED)', 'raw_rate': 0.04})
```

Spot, Futures, and Carry

We also need spot and futures prices to compute carry. We will use Binance as our standard and Deribit as a fallback.

First, we have some simple helpers that make our life a little easier:

```
def http_get_json(url: str, timeout: float = 10.0) -> Optional[dict]:
    txt = fetch_url_text(url, timeout=timeout)
    if not txt:
        return None
    try:
        return json.loads(txt)
    except Exception:
        return None
```

This function makes a request to the given URL and returns the JSON.

```
def deribit_api(path: str, params: Dict[str, str]) -> Optional[dict]:
    q = urllib.parse.urlencode(params)
    url = f”{DERIBIT_BASE}{path}?{q}”
    try:
        with urllib.request.urlopen(url, timeout=10) as resp:
            data = json.loads(resp.read().decode(”utf-8”))
            if data.get(”result”) is not None:
                return data[”result”]
            return None
    except Exception:
        return None
```

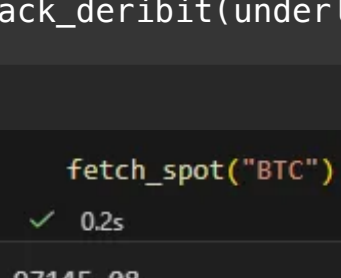
This function simplifies calling the Deribit API.

Next are 2 functions for getting the Binance and Deribit Spot price for a given underlying:

```
def fetch_spot_binance(underlying: str) -> Optional[float]:
    symbol = {”BTC”: “BTCUSDT”, “ETH”:
“ETHUSDT”}.get(underlying.upper())
    if not symbol:
        return None
    url = f”https://api.binance.com/api/v3/ticker/price?symbol={symbol}”
    try:
        with urllib.request.urlopen(url, timeout=10) as resp:
            data = json.loads(resp.read().decode(”utf-8”))
            return float(data[”price”]) if “price” in data else None
    except Exception:
        return None


def fetch_spot_fallback_deribit(underlying: str) -> Optional[float]:
    # Deribit index price for BTC/ETH in USD
    index_name = “btc_usd” if underlying.upper() == “BTC” else “eth_usd”
    res = deribit_api(”public/get_index_price”, {”index_name”:
index_name})
    if res and “index_price” in res:
        return float(res[”index_price”])
    return None
```

As I said before, we want to use Binance as a default and Deribit as a fallback. The following function does this for us:

```
def fetch_spot(underlying: str) -> Optional[float]:
    px = fetch_spot_binance(underlying)
    if px is not None:
        return px
    return fetch_spot_fallback_deribit(underlying)
```

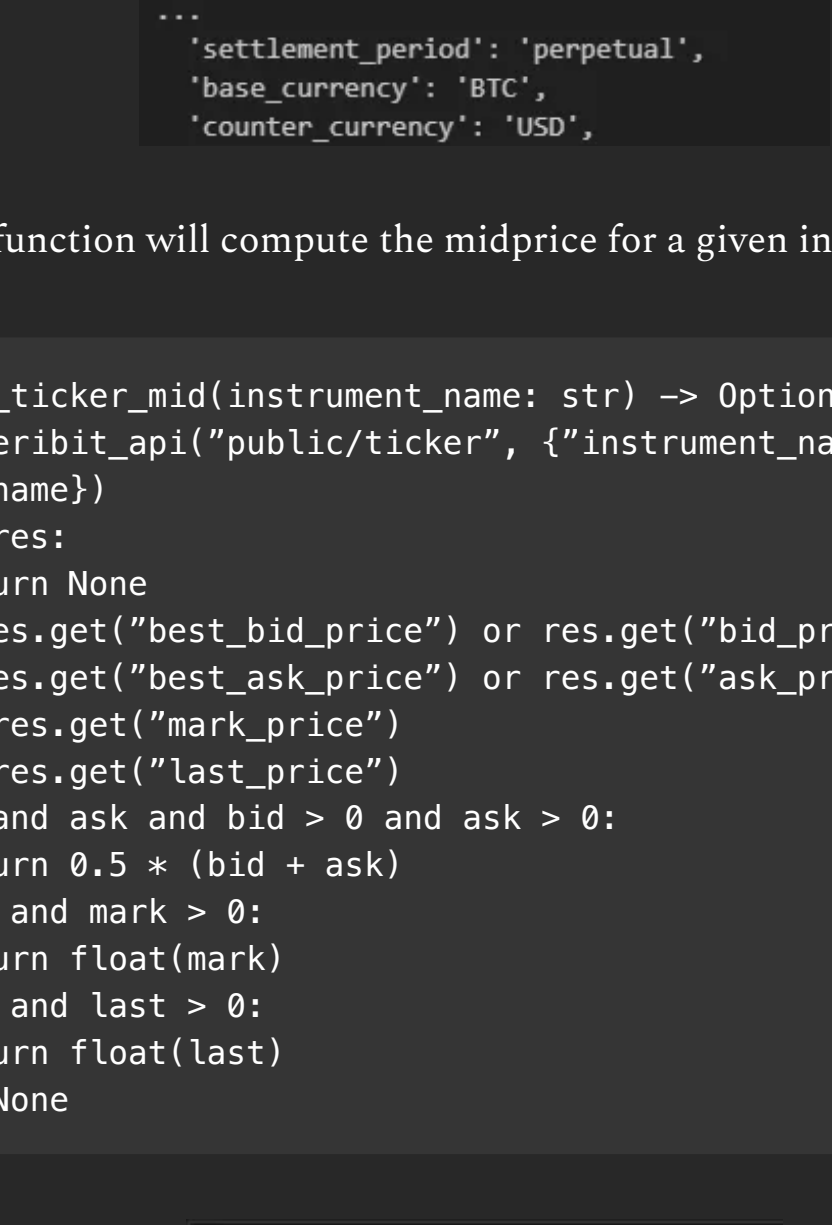


```
fetch_spot("BTC")
✓ 0.2s
97145.08
```

Now that we can grab spot prices, let’s move on to futures. We will use Deribit to get Futures and Option prices, as Deribit is the go-to exchange for those dated derivatives and has by far the highest liquidity and open interest.

The following function will return all non-perp futures on Deribit for the given underlying:

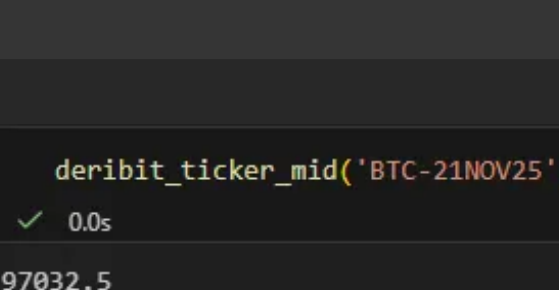
```
def deribit_fetch_futures(currency: str) -> Optional[List[dict]]:
    res = deribit_api(”public/get_instruments”, {”currency”: currency,
“kind”: “future”, “expired”: “false”})
    if not res:
        return None
    return [x for x in res if not x.get(”is_perpetual”, False)]
```



```
deribit_fetch_futures("BTC")
✓ 0.0s
[{'price_index': 'btc_usd',
  'kind': 'future',
  'instrument_name': 'BTC-21NOV25',
  'maker_commission': -0.0001,
  'taker_commission': 0.0005,
  'instrument_type': 'reversed',
  'expiration_timestamp': 1763712000000,
  'creation_timestamp': 1762502415000,
  'is_active': True,
  'contract_size': 10.0,
  'tick_size': 2.5,
  'instrument_id': 547081,
  'min_trade_amount': 10.0,
  'future_type': 'reversed',
  'max_leverage': 25,
  'max_liquidation_commission': 0.0075,
  'block_trade_commission': 0.00025,
  'block_trade_min_trade_amount': 200000,
  'block_trade_tick_size': 0.01,
  'settlement_currency': 'BTC',
  'settlement_period': 'week',
  'base_currency': 'BTC',
  'counter_currency': 'USD',
  'quote_currency': 'USD',
  'tick_size_steps': []},
...
  'settlement_period': 'perpetual',
  'base_currency': 'BTC',
  'counter_currency': 'USD',
```

The following function will compute the midprice for a given instrument:

```
def deribit_ticker_mid(instrument_name: str) -> Optional[float]:
    res = deribit_api(”public/ticker”, {”instrument_name”:
instrument_name})
    if not res:
        return None
    bid = res.get(”best_bid_price”) or res.get(”bid_price”)
    ask = res.get(”best_ask_price”) or res.get(”ask_price”)
    mark = res.get(”mark_price”)
    last = res.get(”last_price”)
    if bid and ask and bid > 0 and ask > 0:
        return 0.5 * (bid + ask)
    if mark and mark > 0:
        return float(mark)
    if last and last > 0:
        return float(last)
    return None
```



```
deribit_ticker_mid('BTC-21NOV25')
✓ 0.0s
97032.5
```

If we can’t get the midprice, we will fall back to the mark price. If that doesn’t work, our last resort is the last traded price.

Why do we even need spot and futures prices? The reason is Carry.
Carry is the cost of keeping a financial position open if the price doesn’t move.
Cost here doesn’t just mean pure P&L. We could have $2 P&L, but if we could have made $5 risk-free instead, then making 2 bucks isn’t all that great.

We can extract the implied carry from the standard futures pricing model:

$$F = Se^{cT}$$

where F is the futures price, S is the spot price, c is carry, and T is the time to maturity.

Solving for c, we obtain:

$$c = \frac{\ln(\frac{F}{S})}{T}$$

If we want the carry implied by a future with maturity of 0.5 years, but we only have futures with maturities of 0.3 years and 0.6 years, then we can calculate the carry for both and interpolate linearly. The following function computes carries and handles everything like requesting prices and interpolating for us:

```
def compute_carry_from_deribit(spot: float, valuation_dt: datetime,
target_T: float, currency: str) -> Tuple[Optional[float], Dict]:
    meta = {”source”: “Deribit”, “used”: [], “interpolation”: “”}
    insts = deribit_fetch_futures(currency)
    if not insts:
        return None, meta
    now_ts = valuation_dt.replace(tzinfo=timezone.utc).timestamp()
    pts = []
    for x in insts:
        exp_ms = x.get(”expiration_timestamp”)
        name = x.get(”instrument_name”)
        if not exp_ms or not name:
            continue
        exp_ts = exp_ms / 1000.0
        T_i = max(0.0, (exp_ts - now_ts) / (365.25 * 24 * 3600.0))
        if T_i <= 1e-6:
            continue
        mid = deribit_ticker_mid(name)
        if not mid or mid <= 0:
            continue
        c_i = math.log(mid / spot) / T_i
        pts.append((T_i, c_i, name))
    if not pts:
        return None, meta
    pts.sort(key=lambda z: z[0])
    lower = None
    upper = None
    for T_i, c_i, name in pts:
        if T_i < target_T:
            lower = (T_i, c_i, name)
        elif T_i >= target_T and upper is None:
            upper = (T_i, c_i, name)
    if lower and upper and upper[0] > lower[0] + 1e-12:
        Tl, cl, nl = lower
        Th, ch, nh = upper
        w = (target_T - Tl) / (Th - Tl)
        c_T = cl + (ch - cl) * w
        meta[”used”] = [{”instrument”: nl, “T”: Tl, “carry”: cl}, {”
instrument”: nh, “T”: Th, “carry”: ch}]
        meta[”interpolation”] = “linear”
        return c_T, meta
    Tn, cn, nn = min(pts, key=lambda z: abs(z[0] - target_T))
    meta[”used”] = [{”instrument”: nn, “T”: Tn, “carry”: cn}]
    meta[”interpolation”] = “nearest”
    return cn, meta
```

We can break carry up into 2 components, risk-free rate and other cashflow like dividends, staking, etc.:

$$c = r - q$$

The following function will compute this q from carry and the risk-free rate:

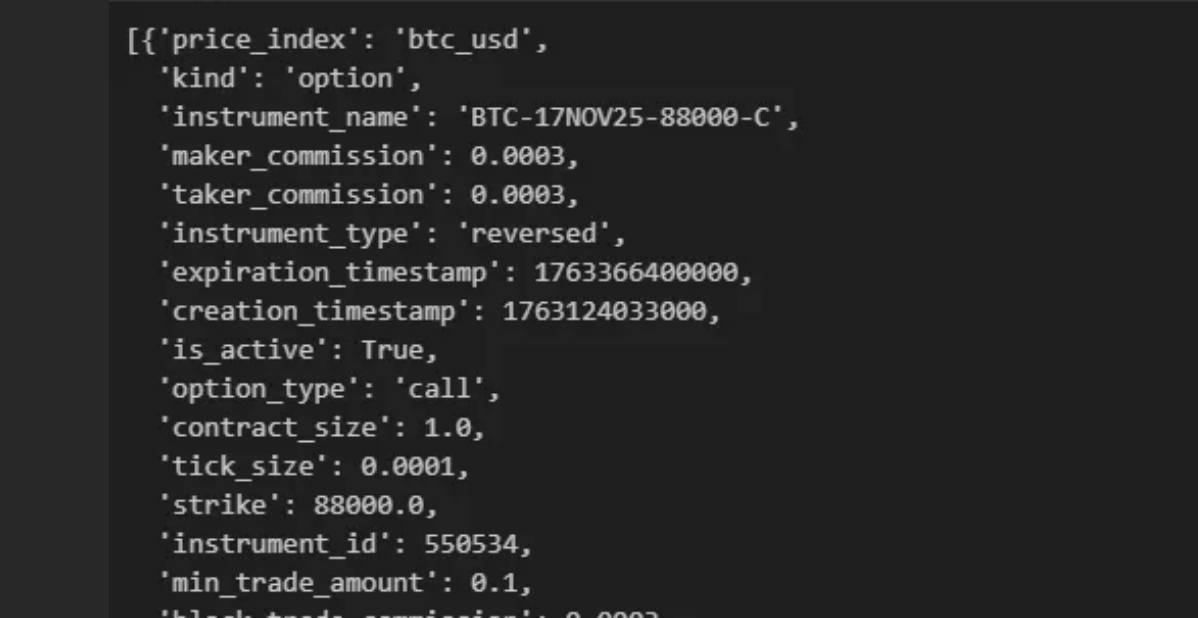
```
def get_rates_auto(spot: float, valuation_dt: datetime, expiry_dt:
datetime, currency: str) -> Tuple[float, float, Dict]:
    T = yearfrac_365(valuation_dt, expiry_dt)
    r_cc, r_meta = get_risk_free_cc(T)
    c_T, c_meta = compute_carry_from_deribit(spot, valuation_dt, T,
currency)
    meta = {
        “r_source”: r_meta.get(”source”, “”),
        “r_raw_rate”: r_meta.get(”raw_rate”, np.nan),
        “c_source”: c_meta.get(”source”, “Deribit”),
        “c_meta”: c_meta,
        “T_years”: T,
    }
    if c_T is None:
        q = 0.0
        meta[”note”] = “Futures basis unavailable; funding set to 0.0”
        return r_cc, q, meta
    q = r_cc - c_T
    return r_cc, float(q), meta
```

Options

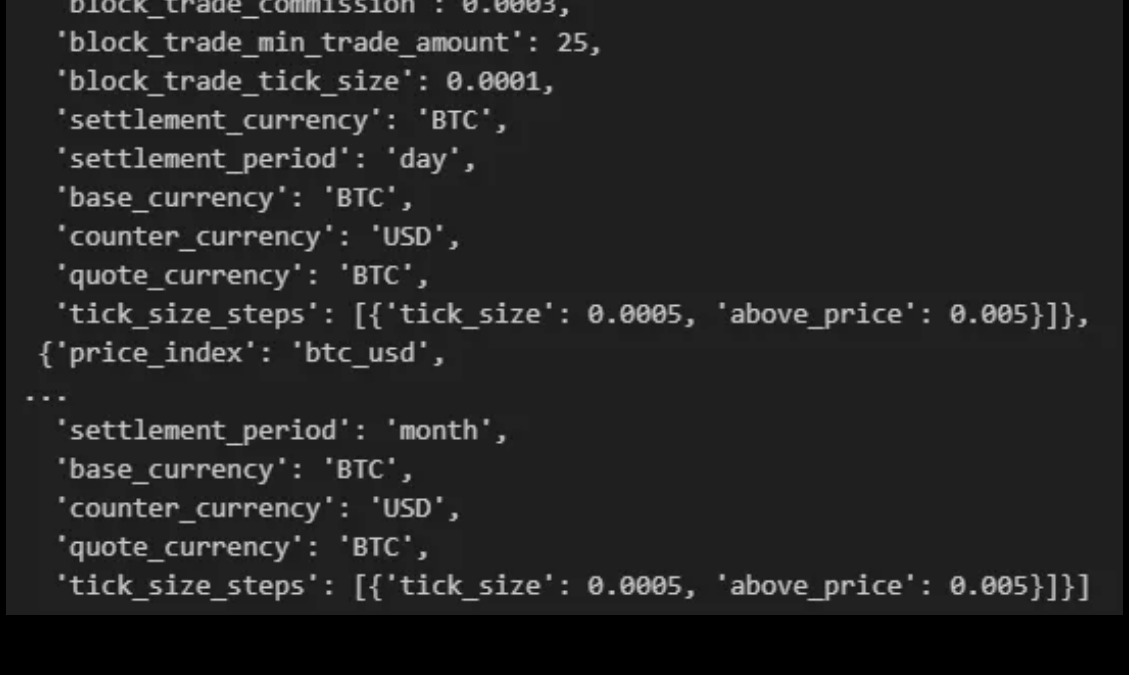
Let’s move on to the last part of data gathering: Option prices and Implied Volatilities.

The following function will grab all live option on Deribit:

```
def deribit_fetch_option_instruments(currency: str) ->
Optional[List[dict]]:
    res = deribit_api(”public/get_instruments”, {”currency”: currency,
“kind”: “option”, “expired”: “false”})
    if not res:
        return None
    return res
```



```
deribit_fetch_option_instruments("BTC")
✓ 0.1s
[{'price_index': 'btc_usd',
  'kind': 'option',
  'instrument_name': 'BTC-17NOV25-88000-C',
  'maker_commission': 0.0003,
  'taker_commission': 0.0003,
  'instrument_type': 'reversed',
  'expiration_timestamp': 1763366400000,
  'creation_timestamp': 1763124033000,
  'is_active': True,
  'option_type': 'call',
  'contract_size': 1.0,
  'tick_size': 0.0001,
  'strike': 88000.0,
  'instrument_id': 550534,
  'min_trade_amount': 0.1,
```



```
  'block_trade_commission': 0.0003,
  'block_trade_min_trade_amount': 25,
  'block_trade_tick_size': 0.0001,
  'settlement_currency': 'BTC',
  'settlement_period': 'day',
  'base_currency': 'BTC',
  'counter_currency': 'USD',
  'quote_currency': 'BTC',
  'tick_size_steps': [{'tick_size': 0.0005, 'above_price': 0.005}]},
 {'price_index': 'btc_usd',
...
  'settlement_period': 'month',
  'base_currency': 'BTC',
  'counter_currency': 'USD',
  'quote_currency': 'BTC',
  'tick_size_steps': [{'tick_size': 0.0005, 'above_price': 0.005}]}]
```

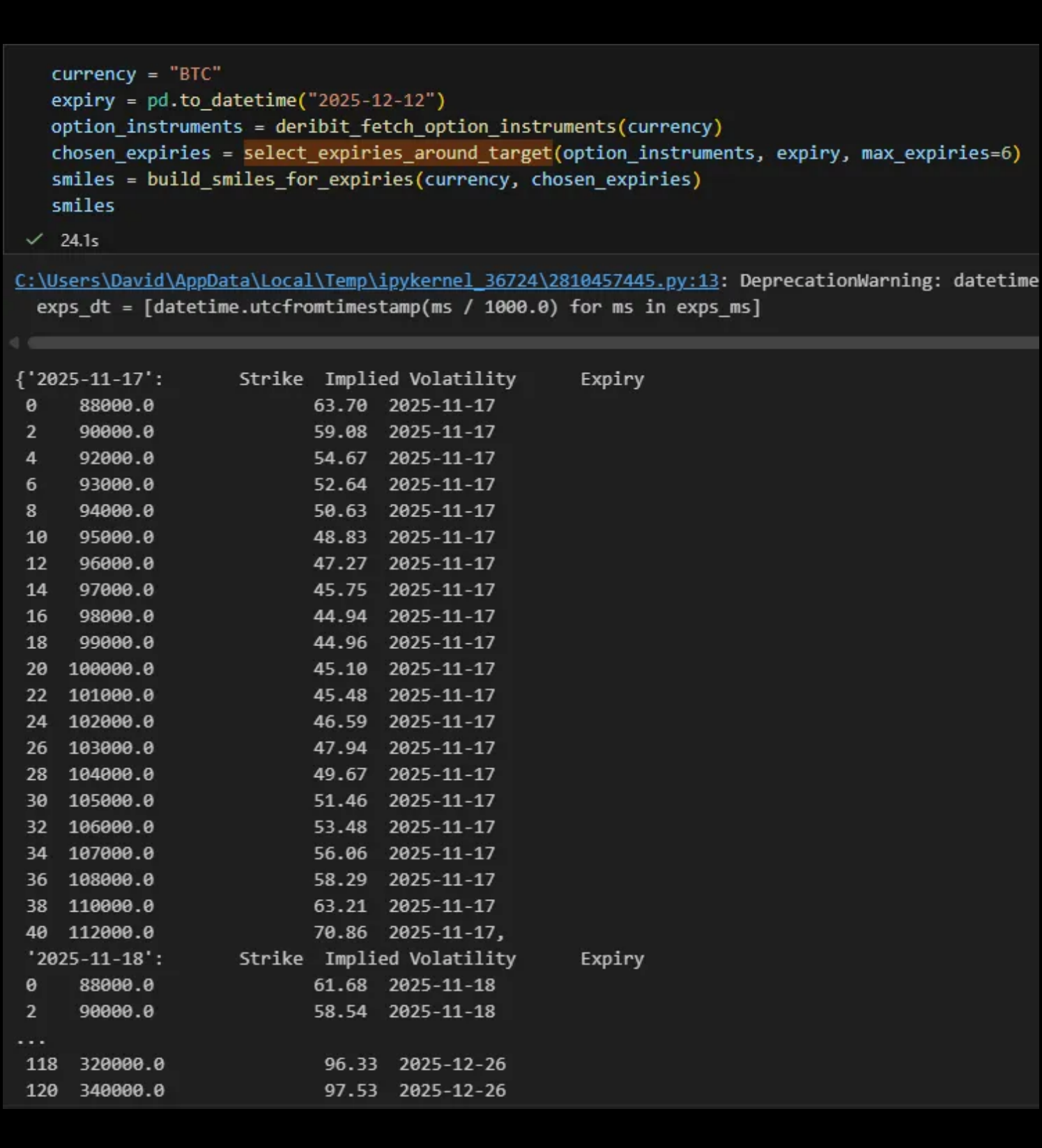
Just like in the futures situation, we only have a finite number of expiries for options.

For a given target expiry, the following function will give us the closest available expiries actually available:

```
def select_expiries_around_target(option_instruments: List[dict],
target_dt: datetime, max_expiries: int = 6) -> List[datetime]:
    # Collect unique expiry datetimes
    exps_ms = sorted(set(int(x[”expiration_timestamp”]) for x in
option_instruments if x.get(”expiration_timestamp”)))
    exps_dt = [datetime.utcfromtimestamp(ms / 1000.0) for ms in exps_ms]
    if not exps_dt:
        return []
    # Rank by distance to target, ensure at least one below and above if
possible
    exps_sorted = sorted(exps_dt, key=lambda d: abs((d -
target_dt).total_seconds()))
    chosen = []
    for d in exps_sorted:
        if d not in chosen:
            chosen.append(d)
        if len(chosen) >= max_expiries:
            break
    # Sort ascending
    chosen = sorted(chosen)
    return chosen
```

For each of those expiries, we can grab all available options and return their implied volatilities. The following function returns a dataframe of all those volatility smiles:

```
def build_smiles_for_expiries(currency: str, chosen_expiries:
List[datetime]) -> Dict[str, pd.DataFrame]:
    ”””
    For each expiry, fetch all option tickers and extract ‘mark_iv’ by
strike.
    Returns a dict: expiry_iso -> DataFrame with columns
[’Strike’,’Implied Volatility’,’Expiry’].
    ‘Implied Volatility’ in percent for easier CSV inspection.
    ”””
    insts = deribit_fetch_option_instruments(currency)
    if not insts:
        return {}
    # Bucket instruments by expiry
    buckets: Dict[int, List[dict]] = {}
    for x in insts:
        exp_ms = x.get(”expiration_timestamp”)
        strike = x.get(”strike”)
        if not exp_ms or strike is None:
            continue
        buckets.setdefault(exp_ms, []).append(x)

    smiles: Dict[str, pd.DataFrame] = {}
    for exp_dt in chosen_expiries:
        exp_ms = int(exp_dt.replace(tzinfo=timezone.utc).timestamp() *
1000)
        group = buckets.get(exp_ms, [])
        rows = []
        for inst in group:
            name = inst.get(”instrument_name”)
            strike = inst.get(”strike”)
            if not name or strike is None:
                continue
            tick = deribit_api(”public/ticker”, {”instrument_name”:
name})
            if not tick:
                continue
            mark_iv = tick.get(”mark_iv”)
            if mark_iv is None or mark_iv <= 0:
                continue
            iv_val = float(mark_iv)
            iv_pct = iv_val * 100.0 if iv_val <= 1.5 else iv_val
            rows.append({”Strike”: float(strike), “Implied Volatility”:
iv_pct})
        if not rows:
            continue
        df = pd.DataFrame(rows).dropna().drop_duplicates(”
Strike”).sort_values(”Strike”)
        df[”Expiry”] = exp_dt.strftime(”%Y-%m-%d”)
        smiles[df[”Expiry”].iloc[0]] = df
    return smiles
```



```
currency = "BTC"
expiry = pd.to_datetime("2025-12-12")
option_instruments = deribit_fetch_option_instruments(currency)
chosen_expiries = select_expiries_around_target(option_instruments, expiry, max_expiries=6)
smiles = build_smiles_for_expiries(currency, chosen_expiries)
smiles
```

```
✓ 24.1s
C:\Users\David\AppData\Local\Temp\ipykernel_36724\2810457445.py:13: DeprecationWarning: datetime
  exps_dt = [datetime.utcfromtimestamp(ms / 1000.0) for ms in exps_ms]

{'2025-11-17':       Strike  Implied Volatility      Expiry
 0    88000.0               63.70  2025-11-17
 2    90000.0               59.08  2025-11-17
 4    92000.0               54.67  2025-11-17
 6    93000.0               52.64  2025-11-17
 8    94000.0               50.63  2025-11-17
 10   95000.0               48.83  2025-11-17
 12   96000.0               47.27  2025-11-17
 14   97000.0               45.75  2025-11-17
 16   98000.0               44.94  2025-11-17
 18   99000.0               44.96  2025-11-17
 20  100000.0               45.10  2025-11-17
 22  101000.0               45.48  2025-11-17
 24  102000.0               46.59  2025-11-17
 26  103000.0               47.94  2025-11-17
 28  104000.0               49.67  2025-11-17
 30  105000.0               51.46  2025-11-17
 32  106000.0               53.48  2025-11-17
 34  107000.0               56.06  2025-11-17
 36  108000.0               58.29  2025-11-17
 38  110000.0               63.21  2025-11-17
 40  112000.0               70.86  2025-11-17,
 '2025-11-18':       Strike  Implied Volatility      Expiry
 0    88000.0               61.68  2025-11-18
 2    90000.0               58.54  2025-11-18
...
 118  320000.0               96.33  2025-12-26
 120  340000.0               97.53  2025-12-26
```

SVI Model fitting

We are finally done with all the prerequisite work. Let’s move on to the fun part of modelling! There are a few problems with using raw option prices for a volatility smile:

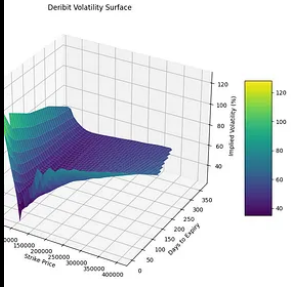
- We only have finitely many strikes.
- Raw prices are noisy.
- Raw prices can violate no-arbitrage conditions.
- You cannot differentiate raw prices.
- Many more problems, mostly related to stability.

That’s why we use SVI. It allows us to model implied volatility (or rather total implied variance, as that is more stable and can be converted back and forth) as a function of strike price (or rather log moneyness for the same reason).

I won’t go too much into detail about this model here, as I have a full dedicated article on this already:

How to Trade the Volatility Smile

VERTOX · 29 JUN



The volatility smile is of great importance to all options traders. Market makers use it to figure out where to quote and volatility traders use it to figure out where volatility will go.

Read full story →

But to summarize, we define log-moneyness k as:

$$k = \ln(\frac{K}{F(T)})$$

where

$$F(T) = Se^{(r-q)T}$$

Total implied variance is defined as:

$$w = T\sigma^2_{\text{imp}}$$

where sigma_imp^2 is implied volatility.

The SVI parameterization is:

$$w(k) = a + b[\rho(k-m) + \sqrt{(k-m)^2 + \sigma^2}]$$

Where a, b, rho, m, and sigma are parameters that you can learn more about in the article above.

The following function computes this w:

```
def svi_total_variance_raw(k: np.ndarray, a: float, b: float, rho:
float, m: float, sigma: float) -> np.ndarray:
    x = k - m
    return a + b * (rho * x + np.sqrt(x * x + sigma * sigma))
```

Our goal is to find the parameters a, b, rho, m, and sigma that best fit the volatility smiles of our actual option implied volatilites (or rather total implied variances).
In the article above, we did this with a simple least-squares. Here, we will be doing the same but in a more robust way, where we start with multiple initial guesses and pick the best resulting fit.

We also put some bounds on our variables:

```
def _svi_bounds():
    # a>=0, b>=0, rho in (-0.999,0.999), m free, sigma>0
    lb = np.array([0.0, 0.0, -0.999, -np.inf, 1e-6])
    ub = np.array([np.inf, np.inf, 0.999, np.inf, np.inf])
    return lb, ub
```

```
def fit_svi_raw(k: np.ndarray, w: np.ndarray) -> Tuple[np.ndarray,
Dict[str, float]]:
    ”””
    Robust least-squares fit to w(k). Returns params [a,b,rho,m,sigma]
and diagnostics.
    ”””
    k = np.asarray(k, dtype=float)
    w = np.asarray(w, dtype=float)
    # Trim obvious outliers/nans
    mask = np.isfinite(k) & np.isfinite(w) & (w > 1e-8) & (w < 50.0)  #
cap huge outliers
    k, w = k[mask], w[mask]
    if k.size < 5:
        raise ValueError(”Not enough data points to fit SVI”)

    lb, ub = _svi_bounds()

    # Initial guesses
    k_med = float(np.median(k))
    w_min = float(np.percentile(w, 10))
    skew = np.corrcoef(k, w)[0, 1] if np.std(k) > 1e-9 and np.std(w) >
1e-9 else 0.0
    guesses = []
    for b0 in [0.1, 0.5, 1.0, 2.0]:
        for rho0 in [-0.7, -0.3, 0.0, 0.3, 0.7]:
            for sig0 in [0.05, 0.15, 0.30]:
                m0 = k_med
                a0 = max(1e-6, w_min * 0.8)
                p0 = np.array([a0, b0, rho0, m0, sig0])
                guesses.append(p0)
    # Bias the sign of rho with skew if available
    if skew < -0.05:
        guesses.append(np.array([w_min, 0.8, -0.6, k_med, 0.2]))
    elif skew > 0.05:
        guesses.append(np.array([w_min, 0.8, 0.6, k_med, 0.2]))

    best = None
    best_cost = np.inf

    def residuals(p):
        a, b, rho, m, sigma = p
        ww = svi_total_variance_raw(k, a, b, rho, m, sigma)
        return ww - w

    for p0 in guesses:
        try:
            res = least_squares(
                residuals,
                x0=p0,
                bounds=(lb, ub),
                loss=”soft_l1”,
                f_scale=np.median(np.abs(w - np.median(w))) + 1e-6,
                max_nfev=5000,
            )
            cost = np.mean((residuals(res.x)) ** 2)
            if np.isfinite(cost) and cost < best_cost:
                best = res
                best_cost = cost
        except Exception:
            continue

    if best is None:
        raise RuntimeError(”SVI fit failed for this expiry”)

    p = best.x
    rmse = float(np.sqrt(best_cost))
    return p, {”rmse_w”: rmse, “n”: int(k.size)}
```

The following function takes a volatility smile for a single expiry as input and handles the conversion of strikes to log moneyness and implied volatility to total implied variance for us, and outputs the resulting new implied volatility smile as a dataframe again:

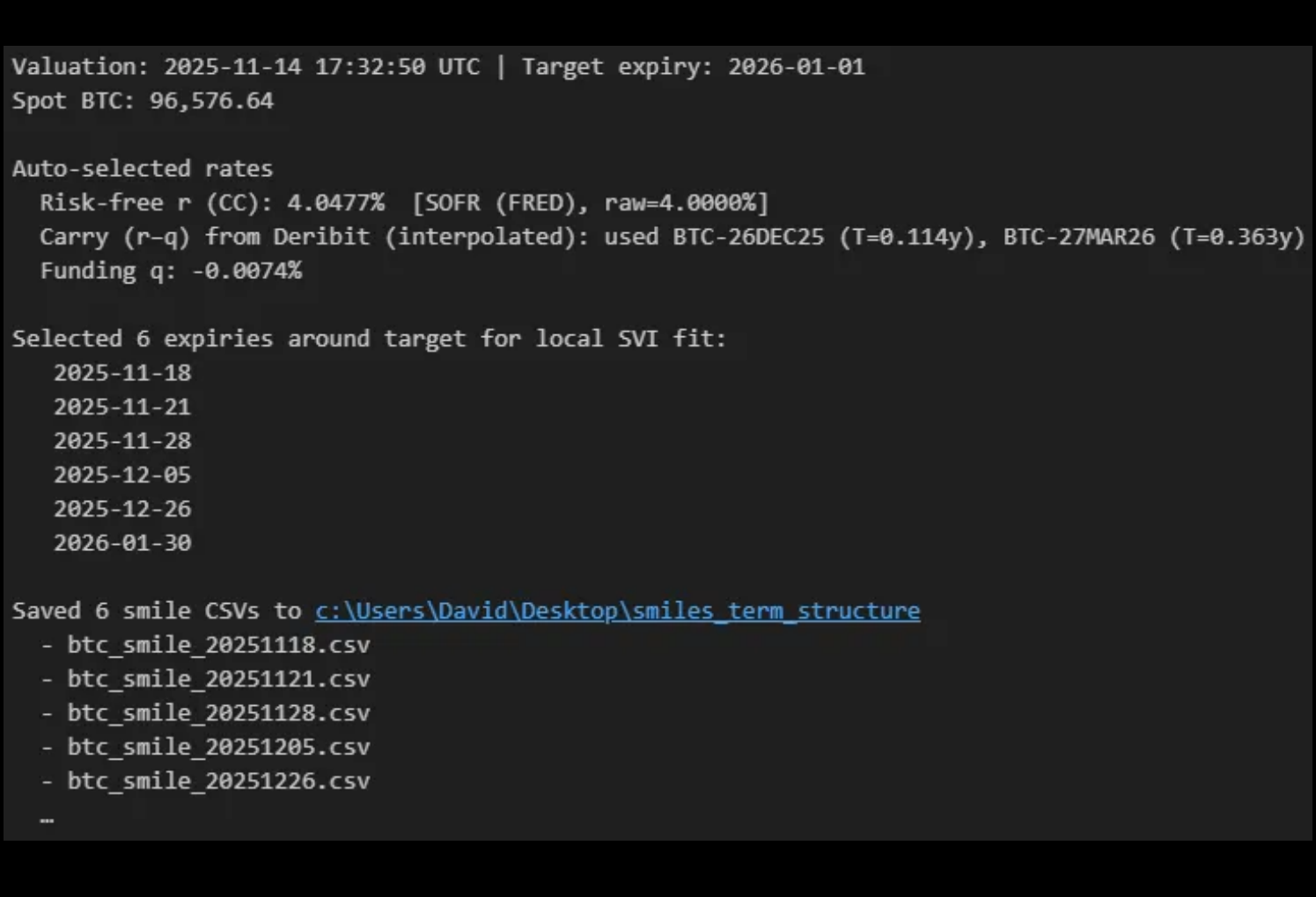
```
def fit_svi_smile_for_expiry(
    spot: float,
    r: float,
    q: float,
    valuation_dt: datetime,
    expiry_dt: datetime,
    df_raw: pd.DataFrame,
) -> Tuple[pd.DataFrame, Dict[str, float]]:
    ”””
    Fit SVI to raw mark IVs for a single expiry and return a smooth
smile DataFrame and diagnostics.
    df_raw columns: Strike, Implied Volatility (percent)
    ”””
    T = yearfrac_365(valuation_dt, expiry_dt)
    if T <= 0:
        raise ValueError(”Expiry already passed”)
    F = spot * np.exp((r - q) * T)

    sdf = df_raw[[”Strike”, “Implied Volatility”]].dropna().copy()
    sdf[”Strike”] = sdf[”Strike”].astype(float)
    # Convert percent -> decimal
    iv_dec = np.clip(pd.to_numeric(sdf[”Implied Volatility”], errors=”
coerce”) / 100.0, 1e-4, 5.0)
    k = np.log(np.clip(sdf[”Strike”].values, 1e-9, 1e12) / F)
    w = np.clip(iv_dec.values ** 2 * T, 1e-8, 50.0)

    # Fit SVI
    params, diag = fit_svi_raw(k, w)
    a, b, rho, m, sigma = params

    # Build smooth strike grid across observed range with modest
extension
    k_lo, k_hi = float(np.min(k)), float(np.max(k))
    k_grid = np.linspace(k_lo - 0.10 * (k_hi - k_lo + 1e-6), k_hi + 0.10
* (k_hi - k_lo + 1e-6), 200)
    w_fit = np.clip(svi_total_variance_raw(k_grid, a, b, rho, m, sigma),
1e-8, 50.0)
    iv_fit = np.sqrt(w_fit / T)
    K_grid = F * np.exp(k_grid)

    out = pd.DataFrame({
        “Strike”: K_grid.astype(float),
        “Implied Volatility”: (iv_fit * 100.0).astype(float),
        “Expiry”: expiry_dt.strftime(”%Y-%m-%d”),
    }).dropna().sort_values(”Strike”)

    diag_out = {”a”: float(a), “b”: float(b), “rho”: float(rho), “m”:
float(m), “sigma”: float(sigma)}
    diag_out.update(diag)
    return out, diag_out
```

And this function does the same for multiple expiries:

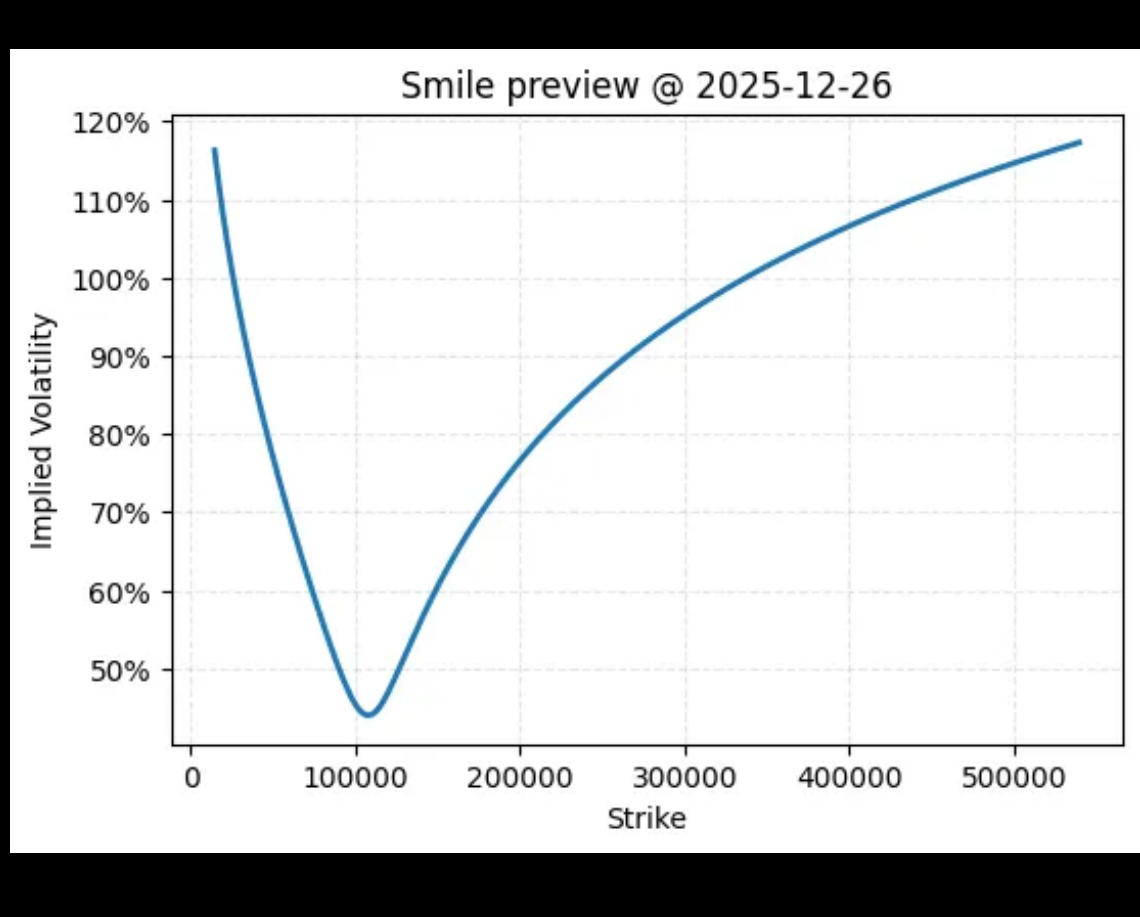
```
def fit_svi_smiles_for_expiries(
    spot: float,
    r: float,
    q: float,
    valuation_dt: datetime,
    expiries: List[datetime],
) -> Tuple[Dict[str, pd.DataFrame], Dict[str, Dict[str, float]]]:
    ”””
    For each expiry, build a raw Deribit smile and fit SVI, returning
fitted smiles and params.
    ”””
    fitted: Dict[str, pd.DataFrame] = {}
    params: Dict[str, Dict[str, float]] = {}
    raw = build_smiles_for_expiries(UNDERLYING, expiries)
    for exp_dt in expiries:
        key = exp_dt.strftime(”%Y-%m-%d”)
        df_raw = raw.get(key)
        if df_raw is None or df_raw.empty:
            continue
        try:
            out_df, diag = fit_svi_smile_for_expiry(spot, r, q,
valuation_dt, exp_dt, df_raw)
            fitted[key] = out_df
            params[key] = diag
        except Exception as e:
            # Skip if fitting fails for this expiry
            continue
    return fitted, params
```

Let’s run everything and look at the resulting smile:

```
# -------- Orchestrate data retrieval & save snapshots --------

SMILE_SOURCE = “svi_local”  # or “deribit”, etc.

# Resolve valuation datetime
if VALUATION_DT is None:
    valuation_dt =
datetime.utcnow().replace(tzinfo=timezone.utc).replace(tzinfo=None)
else:
    valuation_dt = datetime.strptime(VALUATION_DT, “%Y-%m-%d %H:%M”)
expiry_dt = datetime.strptime(EXPIRY_ISO, “%Y-%m-%d”)

print(f”Valuation: {valuation_dt:%Y-%m-%d %H:%M:%S} UTC | Target expiry:
{expiry_dt:%Y-%m-%d}”)

# Spot
spot = fetch_spot(UNDERLYING)
if spot is None:
    raise RuntimeError(”Failed to fetch spot price.”)
print(f”Spot {UNDERLYING}: {spot:,.2f}”)

# Rates (r, q)
r_cc, q_funding, rates_meta = get_rates_auto(spot, valuation_dt,
expiry_dt, UNDERLYING)
print(”\nAuto-selected rates”)
print(f”  Risk-free r (CC): {r_cc:.4%}  [{rates_meta.get(’r_source’)},
raw={rates_meta.get(’r_raw_rate’, float(’nan’)):.4%}]”)
used = rates_meta.get(”c_meta”, {}).get(”used”, [])
if used:
    if len(used) == 2:
        print(
            “  Carry (r-q) from Deribit (interpolated): “
            f”used {used[0][’instrument’]} (T={used[0][’T’]:.3f}y),
{used[1][’instrument’]} (T={used[1][’T’]:.3f}y)”
        )
    else:
        print(
            “  Carry (r-q) from Deribit (nearest): “
            f”used {used[0][’instrument’]} (T={used[0][’T’]:.3f}y)”
        )
else:
    print(”  Carry (r-q): unavailable; funding fallback used.”)
print(f”  Funding q: {q_funding:.4%}”)

# Save rates snapshot
with open(os.path.join(SNAPSHOT_DIR, f”
rates_{UNDERLYING.lower()}_{EXP_STR}.json”), “w”) as f:
    json.dump({”spot”: spot, “r”: r_cc, “q”: q_funding, “meta”:
rates_meta}, f, indent=2)

# Build smiles per expiry
smiles = {}

# 1) Local SVI (preferred): fit SVI to Deribit marks per expiry
if SMILE_SOURCE.lower() in (”svi_local”, “local_svi”, “svi”):
    opt_insts = deribit_fetch_option_instruments(UNDERLYING)
    if not opt_insts:
        raise RuntimeError(”Failed to fetch option instruments from
Deribit.”)
    chosen_expiries = select_expiries_around_target(opt_insts,
expiry_dt, max_expiries=6)
    print(f”\nSelected {len(chosen_expiries)} expiries around target for
local SVI fit:”)
    for d in chosen_expiries:
        print(”  “, d.strftime(”%Y-%m-%d”))
    fitted_smiles, svi_params = fit_svi_smiles_for_expiries(
        spot=spot, r=r_cc, q=q_funding, valuation_dt=valuation_dt,
expiries=chosen_expiries
    )
    smiles = fitted_smiles
    # Save per-expiry SVI params snapshot
    with open(os.path.join(SNAPSHOT_DIR, f”
svi_params_local_{UNDERLYING.lower()}_{EXP_STR}.json”), “w”) as f:
        json.dump(svi_params, f, indent=2)
    # Save raw instruments snapshot (Deribit only)
    with open(os.path.join(SNAPSHOT_DIR, f”
deribit_options_instruments_{UNDERLYING.lower()}_{EXP_STR}.json”), “w”)
as f:
        json.dump(opt_insts, f)


# 2) Raw Deribit marks (no SVI fit)
if SMILE_SOURCE.lower() == “deribit”:
    opt_insts = deribit_fetch_option_instruments(UNDERLYING)
    if not opt_insts:
        raise RuntimeError(”Failed to fetch option instruments from
Deribit.”)
    chosen_expiries = select_expiries_around_target(opt_insts,
expiry_dt, max_expiries=6)
    print(f”\nSelected {len(chosen_expiries)} Deribit expiries around
target:”)
    for d in chosen_expiries:
        print(”  “, d.strftime(”%Y-%m-%d”))
    smiles = build_smiles_for_expiries(UNDERLYING, chosen_expiries)
    with open(os.path.join(SNAPSHOT_DIR, f”
deribit_options_instruments_{UNDERLYING.lower()}_{EXP_STR}.json”), “w”)
as f:
        json.dump(opt_insts, f)

if not smiles:
    raise RuntimeError(”Failed to build any smiles; try a different
target or check connectivity.”)

# Save per-expiry smiles CSVs
saved_files = []
for exp_iso, df in smiles.items():
    df = df[[”Strike”, “Implied Volatility”,
“Expiry”]].dropna().sort_values(”Strike”)
    out_csv = os.path.join(SMILES_DIR, f”
{UNDERLYING.lower()}_smile_{exp_iso.replace(’-’, ’’)}.csv”)
    df.to_csv(out_csv, index=False)
    saved_files.append(out_csv)

print(f”\nSaved {len(saved_files)} smile CSVs to {SMILES_DIR}”)
for p in saved_files[:5]:
    print(”  -”, os.path.basename(p))
if len(saved_files) > 5:
    print(”  …”)

# Quick preview plot for the closest expiry to target
closest_exp = min(smiles.keys(), key=lambda e: abs((datetime.strptime(e,
“%Y-%m-%d”) - expiry_dt).days))
preview = smiles[closest_exp]
plt.figure(figsize=(6,4))
plt.plot(preview[”Strike”], preview[”Implied Volatility”]/100.0, lw=2)
plt.title(f”Smile preview @ {closest_exp}”)
plt.xlabel(”Strike”)
plt.ylabel(”Implied Volatility”)
plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda x, pos: f”
{x:.0%}”))
plt.grid(True, ls=’--’, alpha=0.3)
plt.show()
```

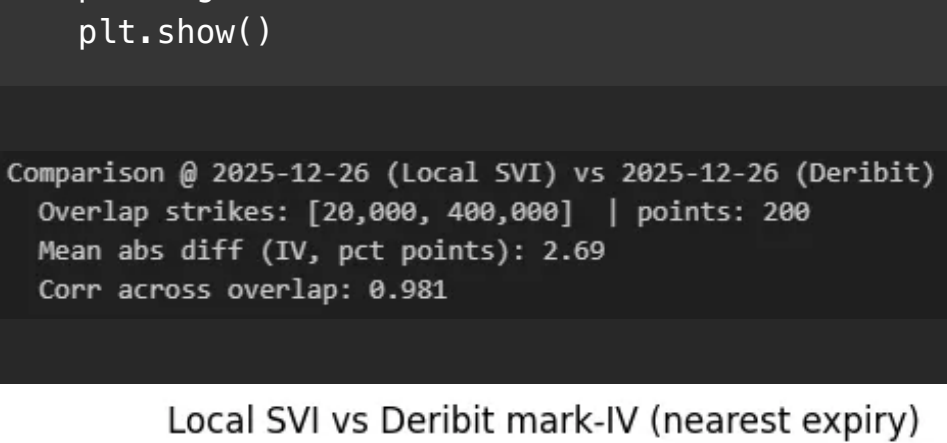


```
Valuation: 2025-11-14 17:32:50 UTC | Target expiry: 2026-01-01
Spot BTC: 96,576.64

Auto-selected rates
  Risk-free r (CC): 4.0477%  [SOFR (FRED), raw=4.0000%]
  Carry (r-q) from Deribit (interpolated): used BTC-26DEC25 (T=0.114y), BTC-27MAR26 (T=0.363y)
  Funding q: -0.0074%

Selected 6 expiries around target for local SVI fit:
   2025-11-18
   2025-11-21
   2025-11-28
   2025-12-05
   2025-12-26
   2026-01-30

Saved 6 smile CSVs to c:\Users\David\Desktop\smiles_term_structure
  - btc_smile_20251118.csv
  - btc_smile_20251121.csv
  - btc_smile_20251128.csv
  - btc_smile_20251205.csv
  - btc_smile_20251226.csv
  …
```

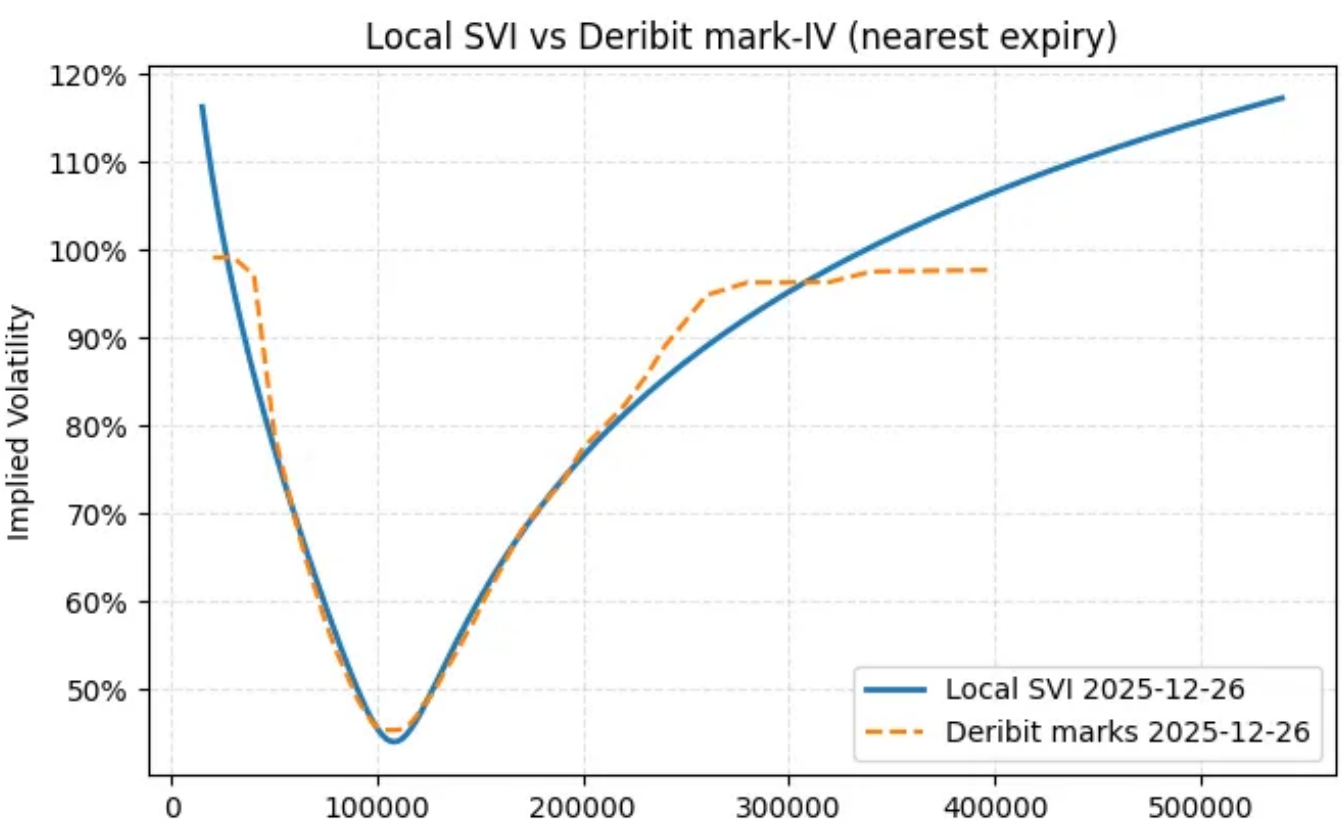


That looks nice and smooth! Now we still need to confirm that it gives us a good fit of real implied volatilities, so let’s overlay them:

```
# Overlay the local SVI smile vs Deribit raw smile, then compare for
nearest common expiry

def _nearest_deribit_exp_for_date(opt_insts: List[dict], target_iso:
str) -> datetime:
    ”””Pick Deribit expiry whose date matches target_iso (YYYY-MM-DD);
if none, nearest by days.”””
    # Build list of datetimes
    exps_ms = sorted(set(int(x[”expiration_timestamp”]) for x in
opt_insts if x.get(”expiration_timestamp”)))
    exps_dt = [datetime.utcfromtimestamp(ms / 1000.0) for ms in exps_ms]
    # Try exact date match first
    for d in exps_dt:
        if d.strftime(”%Y-%m-%d”) == target_iso:
            return d
    # Nearest by absolute days
    tgt = datetime.strptime(target_iso, “%Y-%m-%d”)
    return min(exps_dt, key=lambda d: abs((d - tgt).days))

# Use local SVI CSVs just written as local_smiles
if not smiles:
    print(”Local SVI smiles unavailable for comparison.”)
else:
    # Choose the local expiry nearest to the target expiry
    local_exps = sorted(smiles.keys(), key=lambda e:
abs((datetime.strptime(e, “%Y-%m-%d”) - expiry_dt).days))
    cmp_exp_iso = local_exps[0]
    df_local = smiles[cmp_exp_iso][[”Strike”, “Implied
Volatility”]].dropna().sort_values(”Strike”)

    # Build Deribit smile for the matching/nearest expiry
    opt_insts = deribit_fetch_option_instruments(UNDERLYING)
    if not opt_insts:
        print(”Deribit option instruments unavailable; cannot compare.”)
    else:
        deribit_dt = _nearest_deribit_exp_for_date(opt_insts,
cmp_exp_iso)
        deribit_smiles = build_smiles_for_expiries(UNDERLYING,
[deribit_dt])
        # Find the key (date string)
        deribit_key = next(iter(deribit_smiles.keys())) if
deribit_smiles else None
        if not deribit_key:
            print(”Could not build Deribit smile for comparison.”)
        else:
            df_deri = deribit_smiles[deribit_key][[”Strike”, “Implied
Volatility”]].dropna().sort_values(”Strike”)

            # Interpolate both onto overlapping strike range for
diagnostics
            lo = max(df_local[”Strike”].min(), df_deri[”Strike”].min())
            hi = min(df_local[”Strike”].max(), df_deri[”Strike”].max())
            if hi <= lo:
                print(”No overlapping strike range to compare.”)
            else:
                x = np.linspace(lo, hi, 200)
                local_pct = np.interp(x, df_local[”Strike”].values,
df_local[”Implied Volatility”].values)
                der_pct = np.interp(x, df_deri[”Strike”].values,
df_deri[”Implied Volatility”].values)
                mad = float(np.mean(np.abs(local_pct - der_pct)))
                mad = float(np.mean(np.abs(local_pct - der_pct)))
                corr = float(np.corrcoef(local_pct, der_pct)[0, 1]) if
np.std(local_pct) > 1e-9 and np.std(der_pct) > 1e-9 else float('nan')

                print(f”Comparison @ {cmp_exp_iso} (Local SVI) vs
{deribit_key} (Deribit)”)
                print(f”  Overlap strikes: [{lo:,.0f}, {hi:,.0f}]  |
points: {len(x)}”)
                print(f”  Mean abs diff (IV, pct points): {mad:.2f}”)
                print(f”  Corr across overlap: {corr:.3f}”)

                # Plot overlay
                plt.figure(figsize=(7.5, 4.5))
                plt.plot(df_local[”Strike”], df_local[”Implied
Volatility”]/100.0, lw=2.0, label=f”Local SVI {cmp_exp_iso}”)
                plt.plot(df_deri[”Strike”], df_deri[”Implied
Volatility”]/100.0, lw=1.6, ls=”--”, label=f”Deribit marks
{deribit_key}”)
                plt.title(”Local SVI vs Deribit mark-IV (nearest
expiry)”)
                plt.xlabel(”Strike”)
                plt.ylabel(”Implied Volatility”)
                plt.gca().yaxis.set_major_formatter(FuncFormatter(lambda
v, _: f”{v:.0%}”))
                plt.grid(True, ls=”--”, alpha=0.35)
                plt.legend()
                plt.show()
```



```
Comparison @ 2025-12-26 (Local SVI) vs 2025-12-26 (Deribit)
  Overlap strikes: [20,000, 400,000]  | points: 200
  Mean abs diff (IV, pct points): 2.69
  Corr across overlap: 0.981
```



Looking pretty good! Let’s move on to the next step of our quest to price Polymarket “options”.

Dupire Local Volatility

Alright, so we have an implied volatility surface thanks to our SVI model. What now?

Little spoiler: We are gonna price the options via monte-carlo simulation. This means we are gonna look at a bunch of different price paths, so we can’t just use SVI, which gives us a static volatility surface without any dynamics.

What Dupire Local Volatility does is represent volatility as a function of both time and the current spot price (That’s where it gets its name from!). More precisely, the Dupire Local Volatility model is defined as follows:

$$\sigma^2_{loc}(K,T) = \frac{\frac{\partial C}{\partial T}(K,T)+(e-q)K\frac{\partial C}{\partial K}(K,T)+qC(K,T)}{\frac{1}{2}K^2\frac{\partial^2 C}{\partial K^2}(K,T)}$$

where K is the strike price and C is the price of a call option.
This formula looks a little daunting, but it’s actually not too bad. We already have implied volatility as a function of strike price and time, so all we need to do now is convert our implied volatilities to call prices using Black-Scholes and then differentiate with respect to strike price and time numerically!

This function will calculate the black-scholes call price for us:

```
def bs_call_from_forward(F, K, df, T, sigma):
    if T <= 0:
        return df * max(F - K, 0.0)
    sqrtT = np.sqrt(T)
    vol_sqrtT = sigma * sqrtT
    if vol_sqrtT < 1e-12:
        return df * max(F - K, 0.0)
    d1 = (np.log(F / K) + 0.5 * sigma * sigma * T) / vol_sqrtT
    d2 = d1 - vol_sqrtT
    return df * (F * norm.cdf(d1) - K * norm.cdf(d2))
```

We are gonna make our Implied Volatility Surface that we obtain from SVI into a class to handle it more easily and represent the IV smile for each expiry using a PCHIP interpolator.

```
class ImpliedVolSurface:
    def __init__(self, spot, r, q, Ts, interps):
        self.spot = float(spot)
        self.r = float(r)
        self.q = float(q)
        self.Ts = np.array(Ts, dtype=float)
        self.interps = interps
        self.vol_bounds = (1e-4, 5.0)

    @classmethod
    def from_smile_dict(cls, smiles: Dict[str, pd.DataFrame], spot:
float, r: float, q: float, valuation_dt: datetime):
        entries = []
        for exp_iso, df in smiles.items():
            try:
                expiry_dt = datetime.strptime(exp_iso, “%Y-%m-%d”)
            except Exception:
                continue
            T = yearfrac_365(valuation_dt, expiry_dt)
            if T <= 0:
                continue
            sdf = df[[”Strike”, “Implied Volatility”]].dropna().copy()
            sdf.columns = [”Strike”, “IVpct”]
            sdf = sdf.drop_duplicates(”Strike”).sort_values(”Strike”)
            sdf[”IV”] = sdf[”IVpct”].astype(float) / 100.0
            sdf[”Strike”] = sdf[”Strike”].astype(float)
            entries.append((T, sdf))
        if not entries:
            raise ValueError(”No valid future expiries found.”)
        entries.sort(key=lambda x: x[0])
        Ts = []
        interps = []
        for T, sdf in entries:
            F = spot * np.exp((r - q) * T)
            k = np.log(sdf[”Strike”].values / F)
            vols = sdf[”IV”].values
            interp = PchipInterpolator(k, vols, extrapolate=True)
            Ts.append(T)
            interps.append(interp)
        return cls(spot=spot, r=r, q=q, Ts=np.array(Ts),
interps=interps)

    def forward(self, T):
        return self.spot * np.exp((self.r - self.q) * float(T))

    def _sigma_k_T(self, k, T):
        T = float(T)
        Ts = self.Ts
        if T <= Ts[0]:
            sigma = float(np.clip(self.interps[0](k), *self.vol_bounds))
            return sigma
        if T >= Ts[-1]:
            sigma = float(np.clip(self.interps[-1](k),
*self.vol_bounds))
            return sigma
        idx = np.searchsorted(Ts, T)
        Tl, Th = Ts[idx - 1], Ts[idx]
        wl = np.clip(self.interps[idx - 1](k), *self.vol_bounds) ** 2 *
Tl
        wh = np.clip(self.interps[idx](k), *self.vol_bounds) ** 2 * Th
        w = wl + (wh - wl) * ((T - Tl) / (Th - Tl))
        sigma = np.sqrt(max(w, 1e-12) / T)
        return float(np.clip(sigma, *self.vol_bounds))

    def sigma(self, K, T):
        F = self.forward(T)
        k = np.log(float(K) / F)
        return self._sigma_k_T(k, T)

    def call_price(self, K, T):
        F = self.forward(T)
        df = np.exp(-self.r * float(T))
        sigma = self.sigma(K, T)
        return bs_call_from_forward(F, K, df, T, sigma)

    def tail_probability_from_smile(self, K, T, rel_bump=1e-3):
        T = float(T)
        r = self.r
        eps = max(1e-8, rel_bump * K)
        Kp, Km = K + eps, max(1e-8, K - eps)
        Cp = self.call_price(Kp, T)
        Cm = self.call_price(Km, T)
        dC_dK = (Cp - Cm) / (Kp - Km)
        P = -np.exp(r * T) * dC_dK
        return float(np.clip(P, 0.0, 1.0))
```

Let me explain what the different functions do:

from_smile_dict

This instantiates the class from a dict of volatility smiles that we have generated earlier.

forward

Computes the forward price.

_sigma_k_T

Computes the implied volatility with log moneyness k and maturity T, interpolating linearly in time.

sigma

Computes the implied volatility with strike price K and maturity T by converting K into log moneyness and then using _sigma_k_T.

call_price

Computes the call price with strike price K and maturity T using bs_call_from_forward.

tail_probability_from_smile

This one is gonna be new, it calculates the following (risk-neutral) probability that is obtained from Breeden-Litzenberger:

$$\mathbb{P}(S_T > K) = -e^{rT}\frac{\partial C}{\partial K}(K,T)$$

The probability of ending up above K at expiry. We will be using this to check if our local volatility dynamics and Monte Carlo simulation give us the correct probabilities.

Next, let’s also make our Dupire Local Volatility Surface (Grid of time and spot prices, interpolated with RegularGridInterpolator vs implied volatility) into a class:

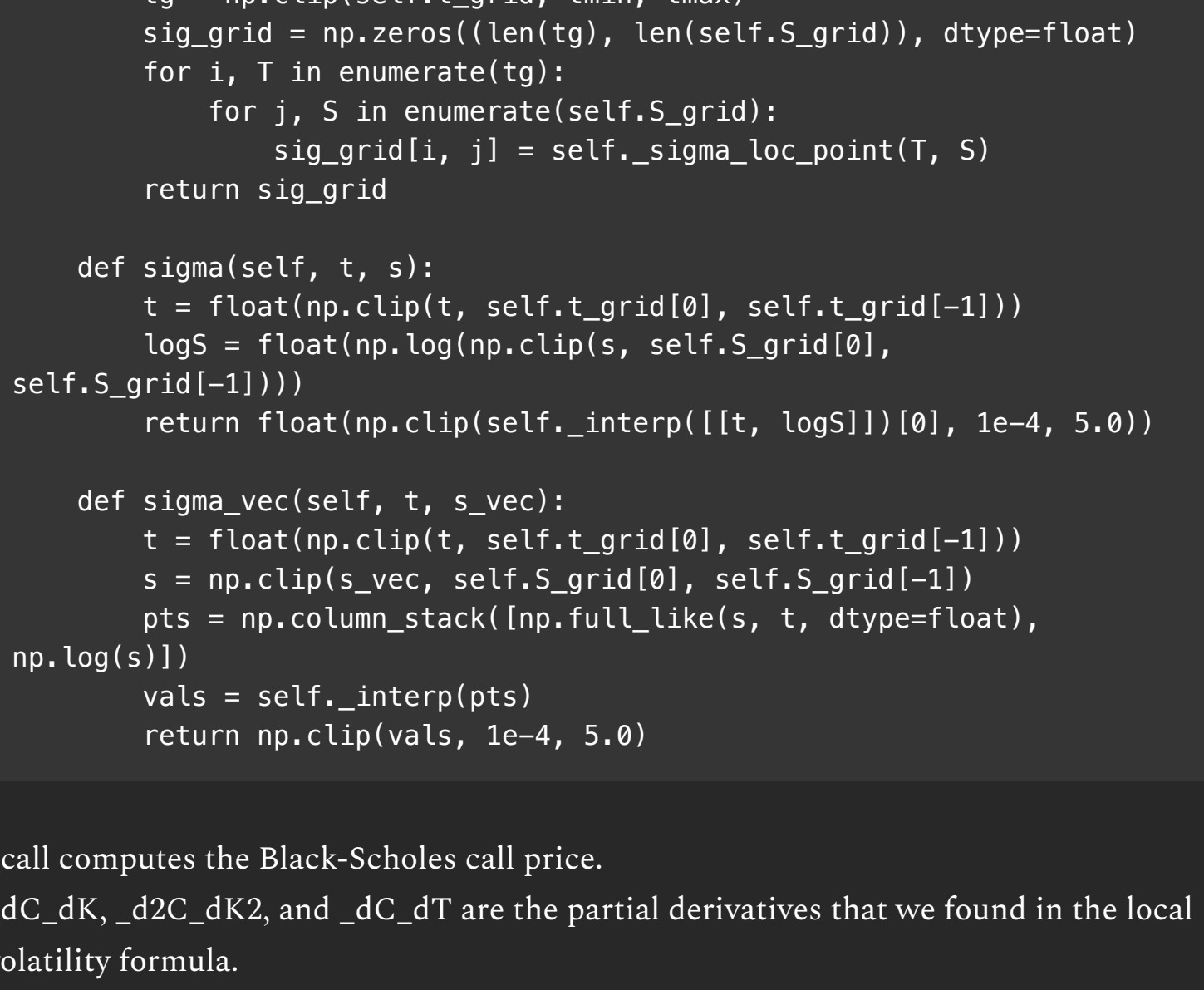
```
class LocalVolSurface:
    def __init__(
        self,
        iv_surface: ImpliedVolSurface,
        T_end: float,
        S_grid: np.ndarray,
        t_grid: np.ndarray,
        dK_rel: float = 5e-3,
        dT_abs: float = 5e-4,
        min_var: float = 1e-8,
        max_var: float = 25.0,
    ):
        self.ivs = iv_surface
        self.T_end = float(T_end)
        self.S_grid = np.asarray(S_grid, dtype=float)
        self.logS_grid = np.log(self.S_grid)
        self.t_grid = np.asarray(t_grid, dtype=float)
        self.dK_rel = dK_rel
        self.dT_abs = dT_abs
        self.min_var = min_var
        self.max_var = max_var
        self._sigma_grid = self._build_sigma_grid()
        self._interp = RegularGridInterpolator(
            (self.t_grid, self.logS_grid), self._sigma_grid,
bounds_error=False, fill_value=None
        )

    # ---- Price derivatives ----
    def _call(self, K, T):
        return self.ivs.call_price(K, T)

    def _dC_dK(self, K, T):
        h = max(1e-8, self.dK_rel * K)
        Cp = self._call(K + h, T)
        Cm = self._call(max(1e-8, K - h), T)
        return (Cp - Cm) / (2.0 * h)

    def _d2C_dK2(self, K, T):
        h = max(1e-8, self.dK_rel * K)
        Cp = self._call(K + h, T)
        C0 = self._call(K, T)
        Cm = self._call(max(1e-8, K - h), T)
        return (Cp - 2.0 * C0 + Cm) / (h * h)

    def _dC_dT(self, K, T):
        h = max(self.dT_abs, 1e-6)
        T_lo = max(self.t_grid[0], T - h)
        T_hi = min(self.t_grid[-1], T + h)
        if T_hi == T_lo:
            T_hi = min(self.t_grid[-1], T + max(1e-6, 0.5 * h))
            T_lo = max(self.t_grid[0], T - max(1e-6, 0.5 * h))
        Cp = self._call(K, T_hi)
        Cm = self._call(K, T_lo)
        return (Cp - Cm) / (T_hi - T_lo)

    def _sigma_loc_point(self, T, K):
        r, q = self.ivs.r, self.ivs.q
        C = self._call(K, T)
        dC_dK = self._dC_dK(K, T)
        d2C_dK2 = self._d2C_dK2(K, T)
        dC_dT = self._dC_dT(K, T)
        num = dC_dT + (r - q) * K * dC_dK + q * C
        den = 0.5 * K * K * max(d2C_dK2, 0.0)
        if den <= 1e-16:
            sigma_fallback = self.ivs.sigma(K, T)
            return float(np.clip(sigma_fallback, 1e-4, 5.0))
        var = np.clip(num / den, self.min_var, self.max_var)
        return float(np.sqrt(var))

    def _build_sigma_grid(self):
        tmin = max(1e-6, min(self.ivs.Ts[0] + self.dT_abs, self.T_end))
        tmax = min(self.T_end, self.ivs.Ts[-1] - self.dT_abs)
        if tmax <= tmin:
            raise ValueError(
                “Time grid cannot be built: ensure expiries bracket the
target maturity.”
            )
        tg = np.clip(self.t_grid, tmin, tmax)
        sig_grid = np.zeros((len(tg), len(self.S_grid)), dtype=float)
        for i, T in enumerate(tg):
            for j, S in enumerate(self.S_grid):
                sig_grid[i, j] = self._sigma_loc_point(T, S)
        return sig_grid

    def sigma(self, t, s):
        t = float(np.clip(t, self.t_grid[0], self.t_grid[-1]))
        logS = float(np.log(np.clip(s, self.S_grid[0],
self.S_grid[-1])))
        return float(np.clip(self._interp([[t, logS]])[0], 1e-4, 5.0))

    def sigma_vec(self, t, s_vec):
        t = float(np.clip(t, self.t_grid[0], self.t_grid[-1]))
        s = np.clip(s_vec, self.S_grid[0], self.S_grid[-1])
        pts = np.column_stack([np.full_like(s, t, dtype=float),
np.log(s)])
        vals = self._interp(pts)
        return np.clip(vals, 1e-4, 5.0)
```

_call computes the Black-Scholes call price.
_dC_dK, _d2C_dK2, and _dC_dT are the partial derivatives that we found in the local volatility formula.
_sigma_loc_point computes this local volatility.
_build_sigma_grid builds a whole grid of local volatilities.
sigma looks up local volatility at (t, s) in our grid.
sigma_vec is a vectorized version of sigma.

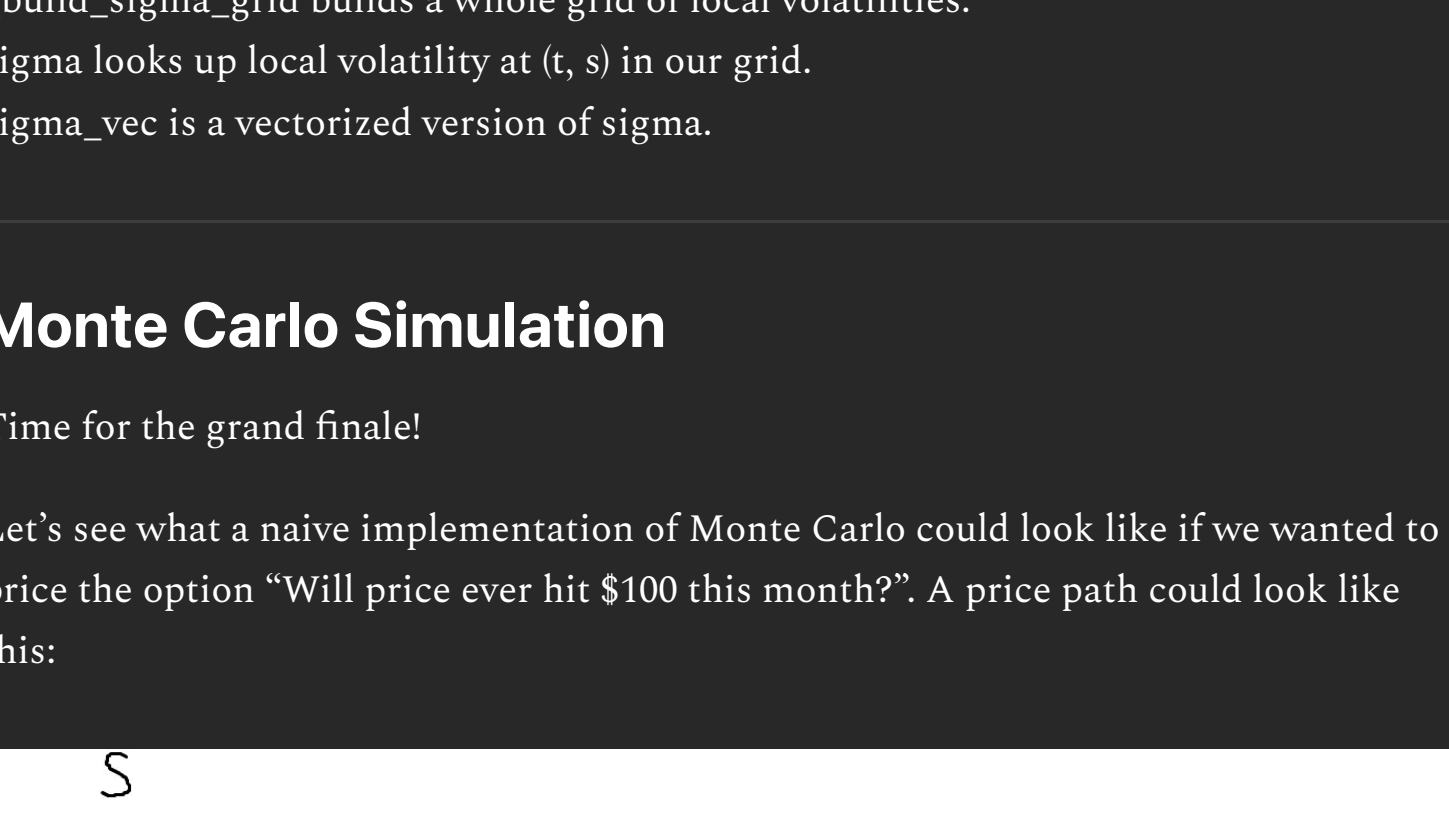
Monte Carlo Simulation

Time for the grand finale!

Let’s see what a naive implementation of Monte Carlo could look like if we wanted to price the option “Will price ever hit $100 this month?”. A price path could look like this:



Looks like we don’t end up hitting $100. Let’s see what could have happened if we used smaller time steps instead:



Now we did end up hitting $100. I think you can see the big problem: Our Monte Carlo Simulation uses discrete timesteps. This underestimates the probability of hitting the target because we could have hit it between any 2 given timesteps.

Luckily, there are 2 ways to go about solving this issue:

Gobet correction:

The idea is that we shift the barrier slightly downwards (or upwards if the barrier is below us).
Define:

$$\text{shift} = c_{Gobet}\sigma\sqrt{\Delta t},\ c_{Gobet} = 0.5826$$

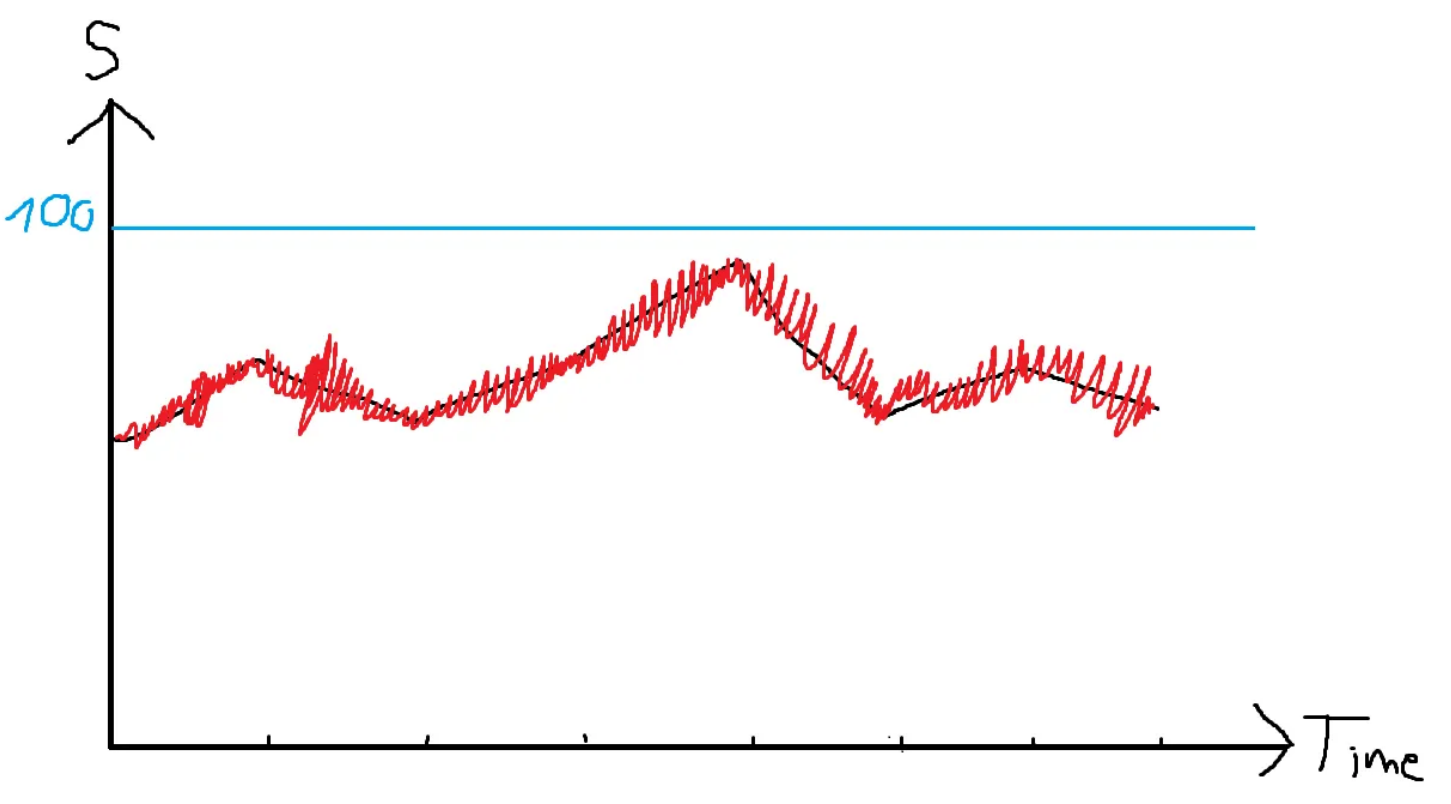
If K was our old barrier, then our new barrier is now:

Up barrier: $Ke^{-\text{shift}}$
Down barrier: Ke^{shift}

Brownian-Bridge:

A Brownian bridge is essentially Brownian motion that starts at some specified point and ends at another specified point.

We can connect 2 prices next to each other in time with a Brownian bridge, which could look something like this:



For paths that didn’t hit the barrier, the probability that the Brownian bridge will cross the barrier is:

$$p_{\text{cross}} = \exp(-\tfrac{2ab}{\text{var}})$$

Where a and b are distances (in log space) from each endpoint to the barrier and

$$\text{var} = \sigma^2\Delta t$$

We will use this to randomly set points that didn’t hit the barrier to “hit” with probability p_cross.

Great, so we now know how to fix the problem with the naive Monte Carlo Simulation, let’s do the actual Monte Carlo Simulation now.

Under our local volatility model:

$$dS_t = (r-q)S_t dt + \sigma_{\text{loc}}(S_t,t)S_t dW_t$$

We are gonna simulate in log space:

$$X_t = \ln S_t$$

with an Euler-Maruyama step:

$$X_{t+\Delta} = X_t + \mu_{\log}(t,S_t)\Delta + \sigma_t\sqrt{\Delta}Z$$
$$\mu_{\log} = (r-q) - \frac{1}{2}\sigma_t^2,\ Z \sim N(0,1)$$

This has some nice properties like ensuring positive prices and some better numerical stability.

Let’s put this all into a class again:

```
class LocalVolMC:
    def __init__(self, iv_surface: ImpliedVolSurface, lv_surface:
LocalVolSurface, seed: int | None = None):
        self.ivs = iv_surface
        self.lvs = lv_surface
        self.r = iv_surface.r
        self.q = iv_surface.q
        if seed is not None:
            np.random.seed(seed)

    def simulate_touch_probability(
        self,
        S0,
        K_barrier,
        T,
        n_paths=200_000,
        n_steps=None,
        mBTCod=”bb”,
        use_mid_sigma=True,
        seed=None,
    ):
        if seed is not None:
            np.random.seed(seed)
        S0 = float(S0)
        T = float(T)
        if n_steps is None:
            n_steps = max(5, int(np.ceil(T * 365)))  # ~daily
        dt = T / n_steps
        sqrt_dt = np.sqrt(dt)

        up_barrier = K_barrier >= S0
        H = np.log(K_barrier)
        gobet_c = 0.5826

        X = np.full(n_paths, np.log(S0), dtype=float)
        hit = np.zeros(n_paths, dtype=bool)

        for j in range(n_steps):
            t = j * dt
            S = np.exp(X)
            sigma_t = self.lvs.sigma_vec(t, S)
            mu_log = (self.r - self.q) - 0.5 * sigma_t * sigma_t
            Z = np.random.standard_normal(n_paths)
            X_next = X + mu_log * dt + sigma_t * sqrt_dt * Z
            S_next = np.exp(X_next)

            if mBTCod in (”gobet”, “bb_gobet”):
                sigma_for_shift = (
                    sigma_t if not use_mid_sigma else
self.lvs.sigma_vec(t + 0.5 * dt, np.sqrt(S * S_next))
                )
                shift = gobet_c * sigma_for_shift * sqrt_dt
                if up_barrier:
                    K_shift = K_barrier * np.exp(-shift)
                    disc_cross = (S >= K_shift) | (S_next >= K_shift)
                else:
                    K_shift = K_barrier * np.exp(+shift)
                    disc_cross = (S <= K_shift) | (S_next <= K_shift)
                hit |= disc_cross
            else:
                if up_barrier:
                    hit |= (S_next >= K_barrier) | (S >= K_barrier)
                else:
                    hit |= (S_next <= K_barrier) | (S <= K_barrier)

            if mBTCod in (”bb”, “bb_gobet”):
                not_hit = ~hit
                if np.any(not_hit):
                    X0 = X[not_hit]
                    X1 = X_next[not_hit]
                    if up_barrier:
                        mask = (X0 < H) & (X1 < H)
                    else:
                        mask = (X0 > H) & (X1 > H)
                    idxs = np.where(not_hit)[0][mask]
                    if idxs.size > 0:
                        X0m = X[idxs]
                        X1m = X_next[idxs]
                        sigma_mid = (
                            self.lvs.sigma_vec(t + 0.5 * dt,
np.sqrt(np.exp(X0m) * np.exp(X1m)))
                            if use_mid_sigma
                            else self.lvs.sigma_vec(t, np.exp(X0m))
                        )
                        var = np.maximum(sigma_mid * sigma_mid * dt, 1e-
12)
                        if up_barrier:
                            a = H - X0m
                            b = H - X1m
                        else:
                            a = X0m - H
                            b = X1m - H
                        p_cross = np.exp(-2.0 * a * b / var)
                        u = np.random.random(size=idxs.size)
                        crossed = u < np.clip(p_cross, 0.0, 1.0)
                        hit[idxs[crossed]] = True

            X = X_next
            if hit.all():
                break

        prob = float(hit.mean())
        lo, hi = wilson_interval(prob, n_paths, confidence=0.95)
        return prob, (lo, hi)

    def simulate_paths(self, S0, T, n_paths=50_000, n_steps=None,
seed=None):
        if seed is not None:
            np.random.seed(seed)
        S0 = float(S0)
        T = float(T)
        if n_steps is None:
            n_steps = max(5, int(np.ceil(T * 365)))
        dt = T / n_steps
        sqrt_dt = np.sqrt(dt)

        X = np.full(n_paths, np.log(S0))
        paths = np.empty((n_paths, n_steps + 1), dtype=float)
        paths[:, 0] = S0
        for j in range(1, n_steps + 1):
            t = (j - 1) * dt
            S = np.exp(X)
            sigma_t = self.lvs.sigma_vec(t, S)
            mu_log = (self.r - self.q) - 0.5 * sigma_t * sigma_t
            Z = np.random.standard_normal(n_paths)
            X = X + mu_log * dt + sigma_t * sqrt_dt * Z
            paths[:, j] = np.exp(X)
        return paths

    def simulate_touch_envelopes(
        self,
        S0,
        T,
        n_paths=200_000,
        n_steps=None,
        mBTCod=”bb”,
        use_mid_sigma=True,
        seed=None,
    ):
        if seed is not None:
            np.random.seed(seed)
        S0 = float(S0)
        T = float(T)
        if n_steps is None:
            n_steps = max(5, int(np.ceil(T ∗ 365)))
        dt = T / n_steps
        sqrt_dt = np.sqrt(dt)
        gobet_c = 0.5826

        X = np.full(n_paths, np.log(S0), dtype=float)
        M_up = np.full(n_paths, S0, dtype=float)
        M_down = np.full(n_paths, S0, dtype=float)

        for j in range(n_steps):
            t = j ∗ dt
            S = np.exp(X)
            sigma_t = self.lvs.sigma_vec(t, S)
            mu_log = (self.r − self.q) − 0.5 ∗ sigma_t ∗ sigma_t
            Z = np.random.standard_normal(n_paths)
            X_next = X + mu_log ∗ dt + sigma_t ∗ sqrt_dt ∗ Z
            S_next = np.exp(X_next)
            sigma_mid = (
                self.lvs.sigma_vec(t + 0.5 ∗ dt, np.sqrt(S ∗ S_next)) if
use_mid_sigma else sigma_t
            )

            if mBTCod in (”gobet”, “bb_gobet”):
                shift = gobet_c ∗ sigma_mid ∗ sqrt_dt
                M_up = np.maximum(M_up, np.maximum(S, S_next) ∗
np.exp(shift))
                M_down = np.minimum(M_down, np.minimum(S, S_next) ∗
np.exp(−shift))
            else:
                M_up = np.maximum(M_up, np.maximum(S, S_next))
                M_down = np.minimum(M_down, np.minimum(S, S_next))

            if mBTCod in (”bb”, “bb_gobet”):
                var = np.maximum(sigma_mid ∗ sigma_mid ∗ dt, 1e−12)
                U = np.clip(np.random.random(n_paths), 1e−12, 1.0 −
1.0e−12)
                c = −0.5 ∗ var ∗ np.log(U)
                delta = np.sqrt((X_next − X) ∗∗ 2 + 4.0 ∗ c)
                H_plus = 0.5 ∗ (X + X_next + delta)
                H_minus = 0.5 ∗ (X + X_next − delta)
                M_up = np.maximum(M_up, np.exp(H_plus))
                M_down = np.minimum(M_down, np.exp(H_minus))

            X = X_next

        ST = np.exp(X)
        return M_up, M_down, ST
```

simulate_touch_probability

This function estimates the probability of touching the barrier at least once by time T using Monte Carlo Simulation and fixing the probability with Gobet and/or Brownian-Bridges.

It also computes Wilson 95% confidence intervals for the touch probability in the end via the following functions:

```
def wilson_interval(prob: float, n: int, confidence: float = 0.95):
    z = norm.ppf((1 + confidence) / 2)
    n = float(n)
    denom = 1 + z ∗ z / n
    center = (prob + z ∗ z / (2 ∗ n)) / denom
    margin = z ∗ np.sqrt(prob ∗ (1 − prob) / n + z ∗ z / (4 ∗ n ∗ n)) /
denom
    return max(0.0, center − margin), min(1.0, center + margin)


def wilson_interval_array(prob: np.ndarray, n: int, confidence: float =
0.95):
    z = norm.ppf((1 + confidence) / 2)
    n = float(n)
    denom = 1.0 + (z ∗ z) / n
    center = (prob + (z ∗ z) / (2.0 ∗ n)) / denom
    margin = z ∗ np.sqrt(prob ∗ (1.0 − prob) / n + (z ∗ z) / (4.0 ∗ n ∗
n)) / denom
    lo = np.clip(center − margin, 0.0, 1.0)
    hi = np.clip(center + margin, 0.0, 1.0)
    return lo, hi
```

simulate_paths

This simply simulates the price paths under local vol, same logic as above.

simulate_touch_envelopes

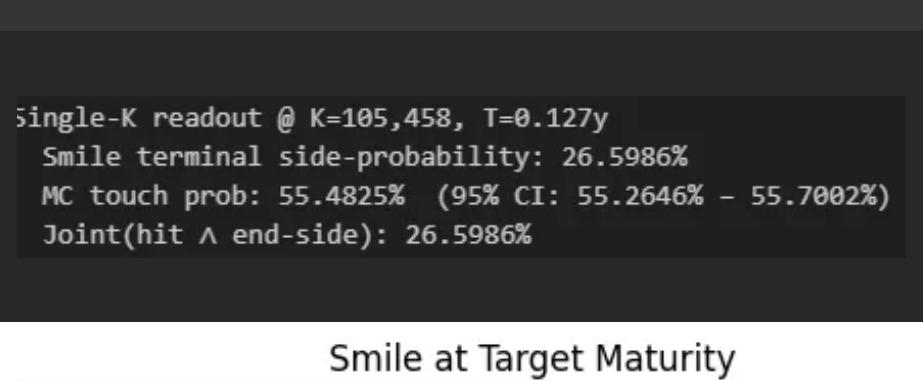
This gives us the running maximum/minimum for each path.
We have the same problem as before: when we have discrete timesteps, we can underestimate maxima and overestimate minima.

It uses the same idea as before and either inflates/deflates the max/min according to Gobet or uses the envelope of a Brownian bridge.

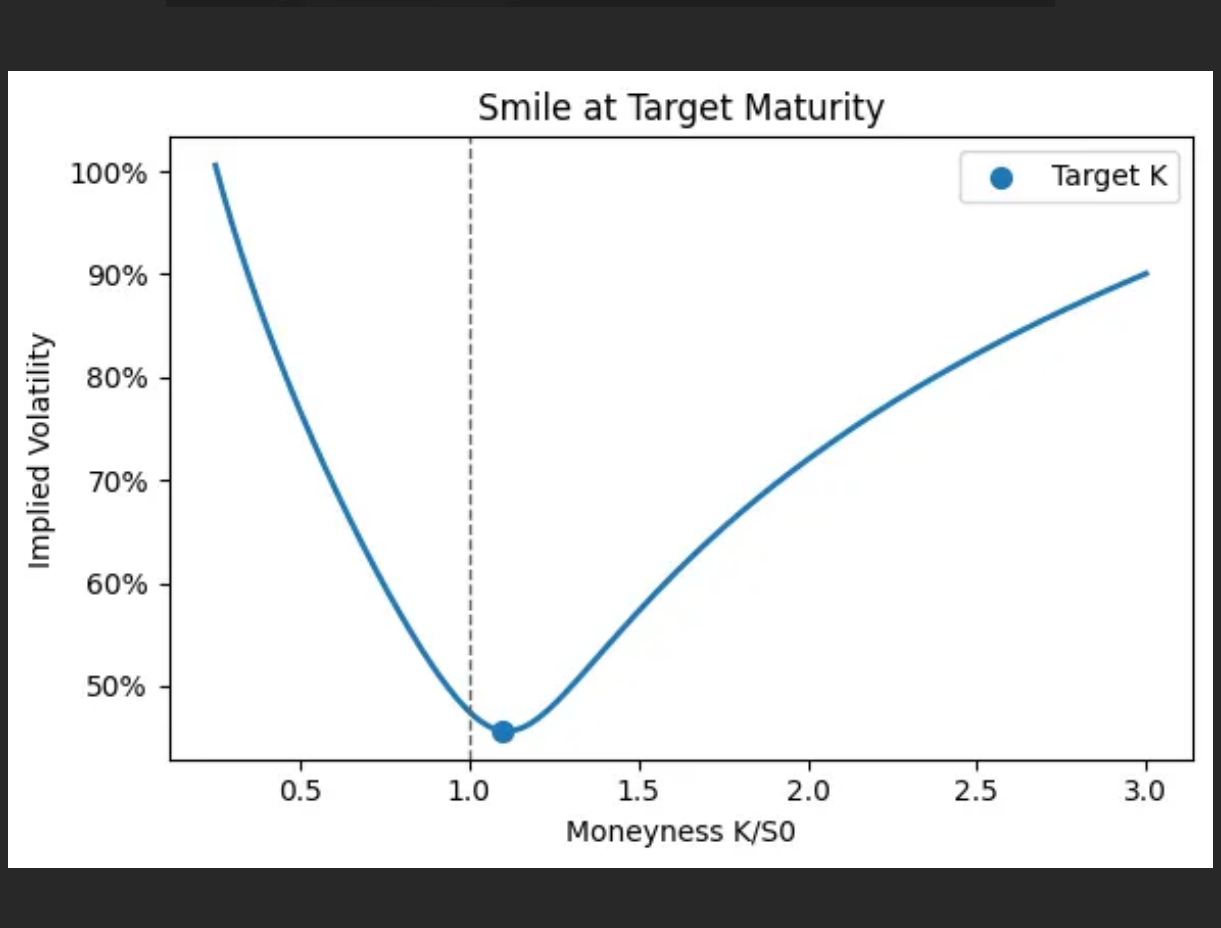
Let's put everything together now! We are gonna:

1. Load previously saved IV smiles
2. Build the implied volatility surface
3. Build the local volatility surface
4. Run Monte Carlo Simulation
5. Output results!

```
# −−−−−−−− Build surfaces for target and run single−K readout −−−−−−−−

# Build IV surface from smiles we saved on disk (load back into dict to
be explicit)
smile_files = [p for p in os.listdir(SMILES_DIR) if p.endswith('.csv')
and p.startswith(UNDERLYING.lower()+”_smile_”)]
smiles_loaded: Dict[str, pd.DataFrame] = {}
for fname in smile_files:
    df = pd.read_csv(os.path.join(SMILES_DIR, fname))
    # determine expiry from filename or column
    m = re.search(r”(\d{8})”, fname)
    if m:
        exp_iso = datetime.strptime(m.group(1), “%Y%m%d”).strftime(”%Y−
%m−%d”)
    elif “Expiry” in df.columns:
        exp_iso = str(df[”Expiry”].iloc[0])[:10]
    else:
        continue
    smiles_loaded[exp_iso] = df

T_years = yearfrac_365(valuation_dt, expiry_dt)
if T_years <= 0:
    raise ValueError(”Target expiry is in the past.”)

ivs = ImpliedVolSurface.from_smile_dict(
    smiles=smiles_loaded,
    spot=spot,
    r=r_cc,
    q=q_funding,
    valuation_dt=valuation_dt,
)

# Local vol grid bounds (independent of target K)
s_lo = S_MIN_FACTOR ∗ spot
s_hi = S_MAX_FACTOR ∗ spot
S_grid = np.exp(np.linspace(np.log(s_lo), np.log(s_hi), 201))

# Time grid within available maturities, trimmed for Dupire bumps
tmin = max(1e−6, ivs.Ts[0] + 5e−4)
tmax = min(T_years, ivs.Ts[−1] − 5e−4)
if tmax <= tmin:
    raise ValueError(
        “Not enough maturity coverage to build local vol. Ensure
expiries bracket the target.”
    )
t_grid = np.linspace(tmin, tmax, 60)

lvs = LocalVolSurface(
    iv_surface=ivs,
    T_end=T_years,
    S_grid=S_grid,
    t_grid=t_grid,
    dK_rel=5e−3,
    dT_abs=5e−4,
    min_var=1e−8,
    max_var=25.0,
)

mc = LocalVolMC(ivs, lvs, seed=SEED)

# Choose a target K for single−K readout: 10% OTM up by default
K_target = float(spot ∗ 1.10)
term_prob_smile = ivs.tail_probability_from_smile(K_target, T_years)
hit_prob, (hit_lo, hit_hi) = mc.simulate_touch_probability(
    S0=spot,
    K_barrier=K_target,
    T=T_years,
    n_paths=N_PATHS_HIT,
    n_steps=N_STEPS,
    mBTCod=MBTCOD,
    use_mid_sigma=USE_MID_SIGMA,
    seed=SEED,
)

if K_target >= spot:
    joint_prob = term_prob_smile  # P(hit ∧ S_T≥K) = P(S_T≥K)
else:
    joint_prob = 1.0 − term_prob_smile

print(f”Single−K readout @ K={K_target:,.0f}, T={T_years:.3f}y”)
print(f”  Smile terminal side−probability: {term_prob_smile:.4%}”)
print(
    f”  MC touch prob: {hit_prob:.4%}  (95% CI: {hit_lo:.4%} −
{hit_hi:.4%}) “
)
print(f”  Joint(hit ∧ end−side): {joint_prob:.4%}”)

# Quick diagnostic plot: smile at target T
strike_range = np.linspace(s_lo, s_hi, 220)
vols_T = np.array([ivs.sigma(k, T_years) for k in strike_range])
plt.figure(figsize=(6,4))
ax1 = plt.subplot(1,1,1)
ax1.plot(strike_range/spot, vols_T, lw=2)
ax1.axvline(1.0, ls='−−', lw=1, color='#666')
ax1.scatter([K_target/spot], [ivs.sigma(K_target, T_years)], s=50,
label='Target K')
ax1.set_title(”Smile at Target Maturity”)
ax1.set_xlabel(”Moneyness K/S0”); ax1.set_ylabel(”Implied Volatility”)
ax1.yaxis.set_major_formatter(FuncFormatter(lambda x, pos: f”{x:.0%}”))
ax1.legend()
plt.tight_layout()
plt.show()
```

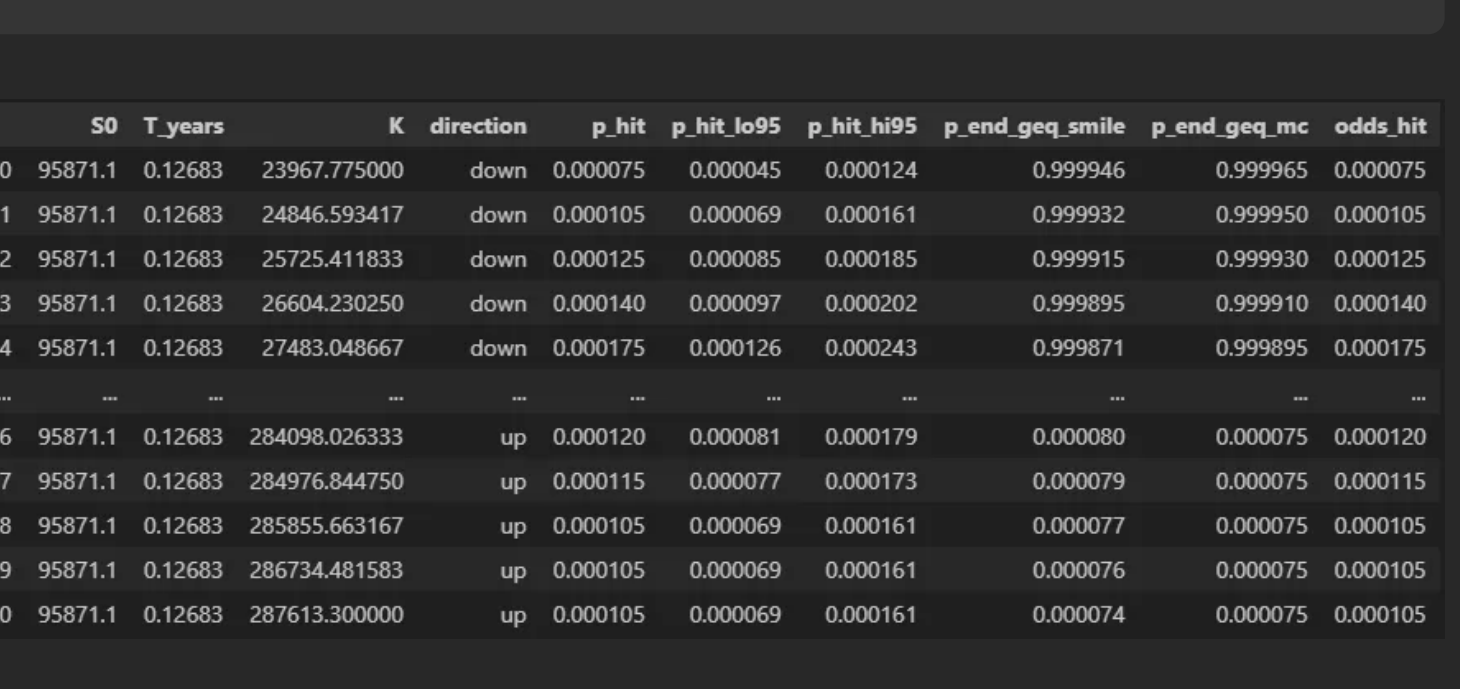


```
Single-K readout @ K=105,458, T=0.127y
  Smile terminal side-probability: 26.5986%
  MC touch prob: 55.4825%  (95% CI: 55.2646% - 55.7002%)
  Joint(hit ∧ end-side): 26.5986%
```



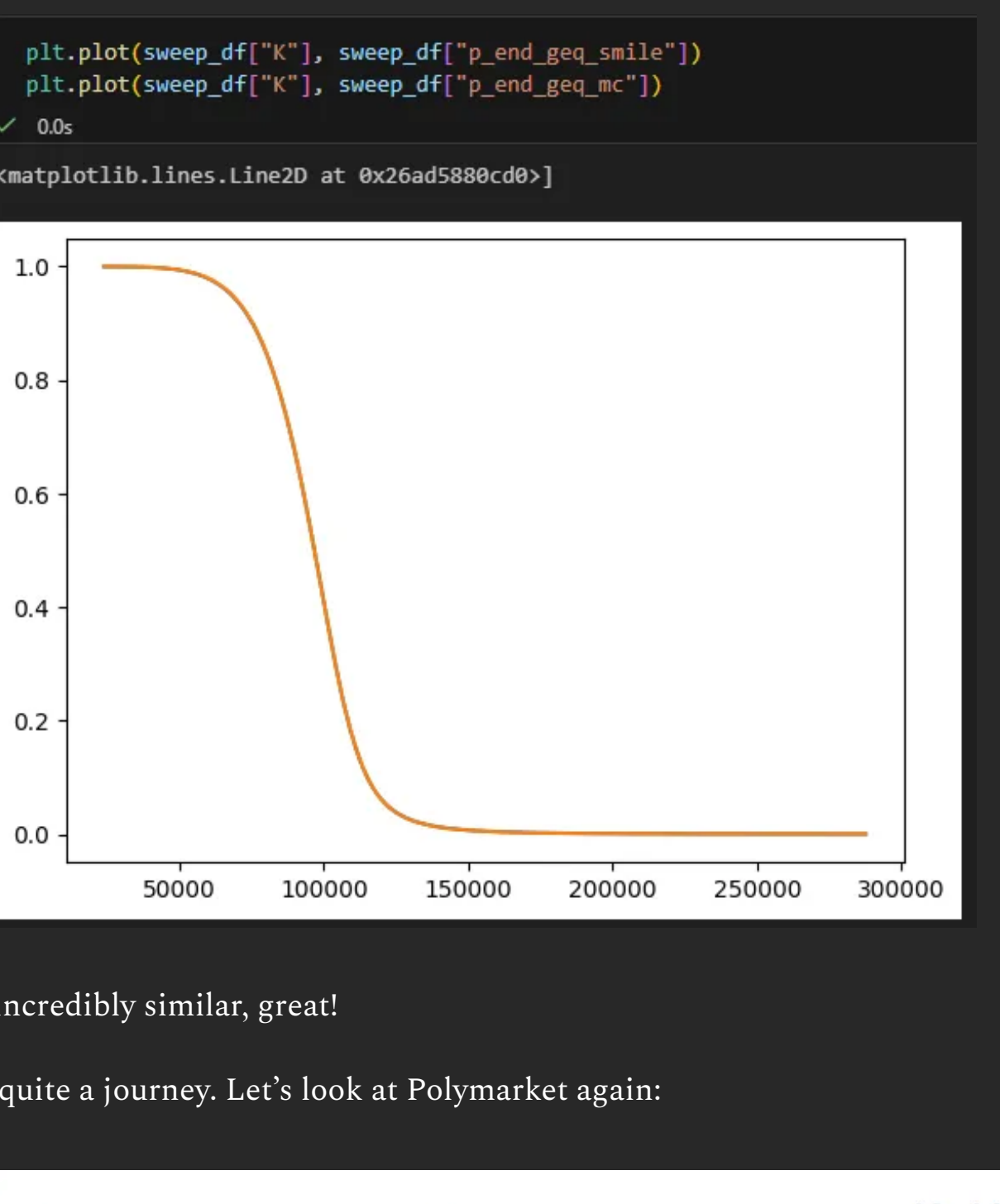
What if we wanted to price different strikes? Do we have to rerun everything each time? The answer is no, thanks to our maxima and minima. We simply generate them once, and then for each strike, we can calculate the probability of hitting this strike price:

```
# Sweep once using envelopes

K_min = s_lo
K_max = s_hi
K_grid = np.linspace(K_min, K_max, SWEEP_POINTS)

M_up, M_down, ST = mc.simulate_touch_envelopes(
    S0=spot,
    T=T_years,
    n_paths=N_PATHS_HIT,
    n_steps=N_STEPS,
    mBTCod=MBTCOD,
    use_mid_sigma=USE_MID_SIGMA,
    seed=SEED,
)

n = M_up.size
M_up_sorted = np.sort(M_up)
M_down_sorted = np.sort(M_down)
ST_sorted = np.sort(ST)

is_up = K_grid >= spot
is_down = ~is_up
p_hit = np.empty_like(K_grid, dtype=float)
lo = np.empty_like(K_grid, dtype=float)
hi = np.empty_like(K_grid, dtype=float)

if np.any(is_up):
    idx = M_up_sorted.searchsorted(K_grid[is_up], side=”left”)
    succ = n − idx
    p = succ / n
    p_hit[is_up] = p
    lo_up, hi_up = wilson_interval_array(p, n, confidence=0.95)
    lo[is_up], hi[is_up] = lo_up, hi_up

if np.any(is_down):
    idx = M_down_sorted.searchsorted(K_grid[is_down], side=”right”)
    p = idx / n
    p_hit[is_down] = p
    lo_dn, hi_dn = wilson_interval_array(p, n, confidence=0.95)
    lo[is_down], hi[is_down] = lo_dn, hi_dn

p_end_geq_smile = np.array([ivs.tail_probability_from_smile(K, T_years)
for K in K_grid])
idx_end = ST_sorted.searchsorted(K_grid, side=”left”)
p_end_geq_mc = (n − idx_end) / n

sweep_df = pd.DataFrame(
    {
        “K”: K_grid,
        “direction”: np.where(is_up, “up”, “down”),
        “p_hit”: p_hit,
        “p_hit_lo95”: lo,
        “p_hit_hi95”: hi,
        “p_end_geq_smile”: p_end_geq_smile,
        “p_end_geq_mc”: p_end_geq_mc,
        “odds_hit”: p_hit / np.maximum(1.0 − p_hit, 1e−12),
    }
)

# Insert minimal config columns for downstream use
sweep_df.insert(0, “S0”, spot)
sweep_df.insert(1, “T_years”, T_years)

sweep_df.to_csv(SWEEP_CSV, index=False)
print(f”Saved sweep CSV to: {SWEEP_CSV}”)

meta_json = {
    “spot”: float(spot),
    “expiry_iso”: expiry_dt.strftime(”%Y−%m−%d”),
    “valuation_dt_iso”: valuation_dt.strftime(”%Y−%m−%d %H:%M:%S”),
    “T_years”: float(T_years),
    “r”: float(r_cc),
    “q”: float(q_funding),
    “smiles_dir”: os.path.abspath(SMILES_DIR),
    “mBTCod”: MBTCOD,
    “use_mid_sigma”: bool(USE_MID_SIGMA),
    “n_paths”: int(N_PATHS_HIT),
    “n_steps”: (int(np.ceil(T_years ∗ 365)) if N_STEPS is None else
int(N_STEPS)),
    “seed”: int(SEED),
    “S_grid_min”: float(s_lo),
    “S_grid_max”: float(s_hi),
    “S_grid_points”: int(len(S_grid)),
    “t_grid_points”: int(len(t_grid)),
    “rates_meta”: rates_meta,
}
with open(META_JSON, “w”) as f:
    json.dump(meta_json, f, indent=2)
print(f”Saved sweep meta JSON to: {META_JSON}”)
```



	S0	T_years	K	direction	p_hit	p_hit_lo95	p_hit_hi95	p_end_geq_smile	p_end_geq_mc	odds_hit
0	95871.1	0.12683	23967.775000	down	0.000075	0.000045	0.000124	0.999946	0.999965	0.000075
1	95871.1	0.12683	24846.593417	down	0.000105	0.000069	0.000161	0.999932	0.999950	0.000105
2	95871.1	0.12683	25725.411833	down	0.000125	0.000085	0.000185	0.999915	0.999930	0.000125
3	95871.1	0.12683	26604.230250	down	0.000140	0.000097	0.000202	0.999895	0.999910	0.000140
4	95871.1	0.12683	27483.048667	down	0.000175	0.000126	0.000243	0.999871	0.999895	0.000175
...	...	...	...	...	...	...	...	...	...	...
296	95871.1	0.12683	284098.026333	up	0.000120	0.000081	0.000179	0.000080	0.000075	0.000120
297	95871.1	0.12683	284976.844750	up	0.000115	0.000077	0.000173	0.000079	0.000075	0.000115
298	95871.1	0.12683	285855.663167	up	0.000105	0.000069	0.000161	0.000077	0.000075	0.000105
299	95871.1	0.12683	286734.481583	up	0.000105	0.000069	0.000161	0.000076	0.000075	0.000105
300	95871.1	0.12683	287613.300000	up	0.000105	0.000069	0.000161	0.000074	0.000075	0.000105

This is where the risk-neutral Breeden-Litzenberger probability from earlier comes in. p_end_qeq_smile is this theoretical probability, and p_end_geq_mc is the probability we obtain from our Monte Carlo Simulation. Let's plot them together:

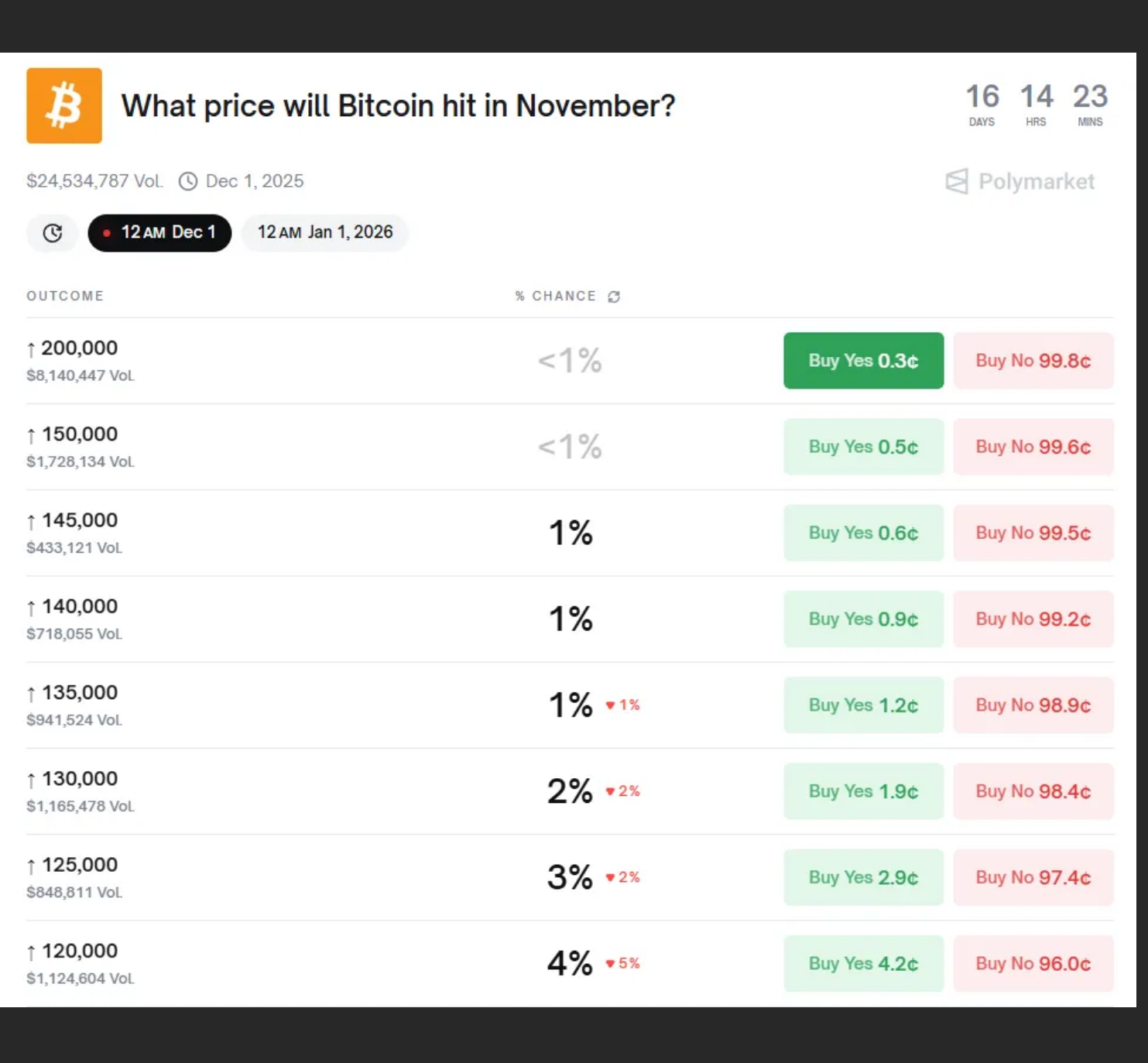


```
plt.plot(sweep_df["K"], sweep_df["p_end_geq_smile"])
plt.plot(sweep_df["K"], sweep_df["p_end_geq_mc"])
```

```
✓ 0.0s
[<matplotlib.lines.Line2D at 0x26ad5880cd0>]
```

They are incredibly similar, great!

That was quite a journey. Let's look at Polymarket again:



What price will Bitcoin hit in November?

16 DAYS 14 HRS 23 MINS

$24,534,787 Vol. Dec 1, 2025

Polymarket

12 AM Dec 1 | 12 AM Jan 1, 2026

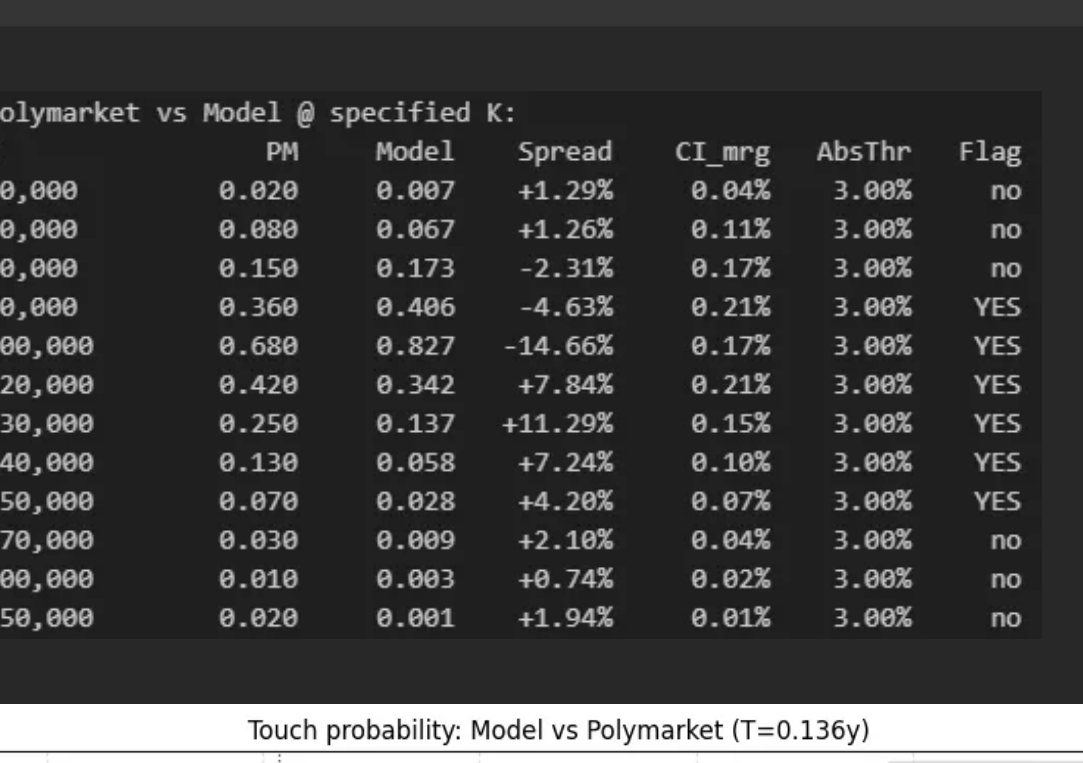
OUTCOME	% CHANCE		
↑ 200,000 $8,140,447 Vol.	<1%	Buy Yes 0.3¢	Buy No 99.8¢
↑ 150,000 $1,728,134 Vol.	<1%	Buy Yes 0.5¢	Buy No 99.6¢
↑ 145,000 $433,121 Vol.	1%	Buy Yes 0.6¢	Buy No 99.5¢
↑ 140,000 $718,055 Vol.	1%	Buy Yes 0.9¢	Buy No 99.2¢
↑ 135,000 $941,524 Vol.	1% ▾1%	Buy Yes 1.2¢	Buy No 98.9¢
↑ 130,000 $1,165,478 Vol.	2% ▾2%	Buy Yes 1.9¢	Buy No 98.4¢
↑ 125,000 $848,811 Vol.	3% ▾2%	Buy Yes 2.9¢	Buy No 97.4¢
↑ 120,000 $1,124,604 Vol.	4% ▾5%	Buy Yes 4.2¢	Buy No 96.0¢

After reading this article, it shouldn't be difficult to say how likely we are to hit, say, $130.000 in November.

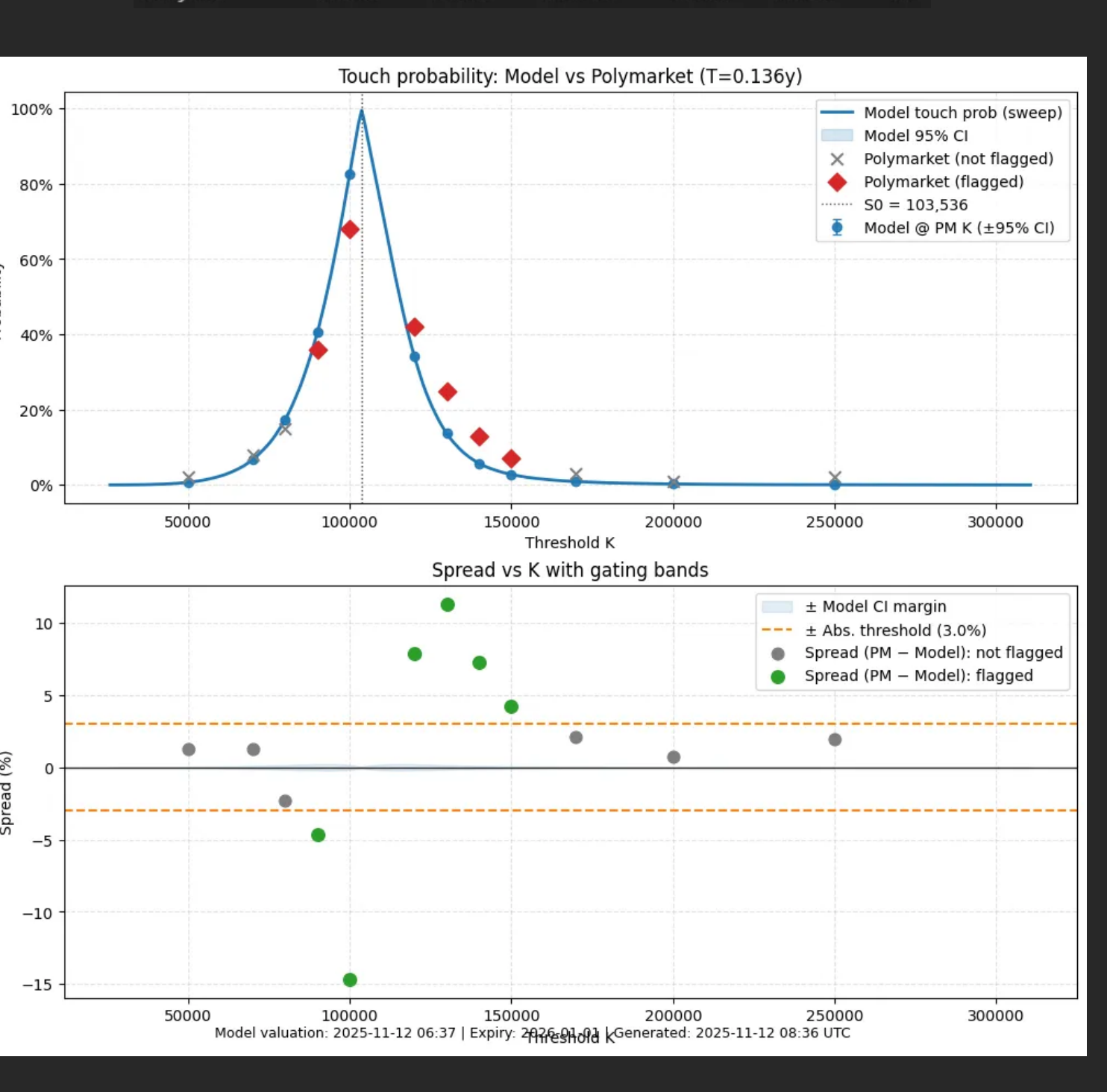
Whenever the probability implied by Polymarket is significantly different than what is implied by Deribit, we have an opportunity to trade.

You can compare them visually using this piece of code:

```
# Polymarket input: list or optional CSV
from typing import Sequence, Tuple

# Option A: edit in−place here
POLYMARKET_QUOTES: List[Tuple[float, float]] = [
    # (K, probability in [0,1])
    (250000.0, 0.02),
    (200000.0, 0.01),
    (170000.0, 0.03),
    (150000.0, 0.07),
    (140000.0, 0.13),
    (130000.0, 0.25),
    (120000.0, 0.42),
    (100000.0, 0.68),
    (90000.0, 0.36),
    (80000.0, 0.15),
    (70000.0, 0.08),
    (50000.0, 0.02),
]

# Option B: load from CSV (uncomment and set path)
# pm_df = pd.read_csv(”polymarket_quotes.csv”)
# POLYMARKET_QUOTES = [(float(k), float(p)) for k, p in zip(pm_df['K'],
pm_df['p'])]

pm_K = np.array([k for k, _ in POLYMARKET_QUOTES], dtype=float)
pm_p = np.array([p for _, p in POLYMARKET_QUOTES], dtype=float)

# Interpolate model values and CI at Polymarket thresholds
model_p_at_pmK = np.interp(pm_K, sweep_df[”K”].values, sweep_df[”
p_hit”].values)
lo_at_pmK = np.interp(pm_K, sweep_df[”K”].values, sweep_df[”
p_hit_lo95”].values)
hi_at_pmK = np.interp(pm_K, sweep_df[”K”].values, sweep_df[”
p_hit_hi95”].values)
ci_margin_at_pmK = np.minimum(model_p_at_pmK − lo_at_pmK, hi_at_pmK −
model_p_at_pmK)
ci_margin_at_pmK = np.clip(ci_margin_at_pmK, 0.0, 1.0)

spreads = pm_p − model_p_at_pmK
spreads_pct = spreads ∗ 100.0
ci_margin_pct_at_pmK = ci_margin_at_pmK ∗ 100.0
abs_thresh_pct = ABS_THRESHOLD ∗ 100.0
gate_threshold_pct = np.maximum(ci_margin_pct_at_pmK, abs_thresh_pct)
flagged = np.abs(spreads_pct) > gate_threshold_pct

# Summary table printout
if pm_K.size > 0:
    print(”\nPolymarket vs Model @ specified K:”)
    header = (
        “K”.ljust(12)
        + “PM”.rjust(8)
        + “Model”.rjust(10)
        + “Spread”.rjust(10)
        + “CI_mrg”.rjust(10)
        + “AbsThr”.rjust(9)
        + “Flag”.rjust(7)
    )
    print(header)
    for i in np.argsort(pm_K):
        print(
            f”{pm_K[i]:,.0f}”.ljust(12)
            + f”{pm_p[i]:.3f}”.rjust(8)
            + f”{model_p_at_pmK[i]:.3f}”.rjust(10)
            + f”{spreads_pct[i]:+.2f}%”.rjust(10)
            + f”{ci_margin_pct_at_pmK[i]:.2f}%”.rjust(10)
            + f”{abs_thresh_pct:.2f}%”.rjust(9)
            + (” YES” if flagged[i] else “  no”).rjust(7)
        )

# Plots
fig, (ax1, ax2) = plt.subplots(2, 1, figsize=(10, 9),
constrained_layout=True)

# Panel 1: model curve + CI, model−at−PM−K error bars, PM points
ax1.plot(sweep_df[”K”], sweep_df[”p_hit”], lw=2, label=”Model touch prob
(sweep)”, color=”#1f77b4”)
ax1.fill_between(sweep_df[”K”], sweep_df[”p_hit_lo95”], sweep_df[”
p_hit_hi95”], color=”#1f77b4”, alpha=0.18, label=”Model 95% CI”)

# Error bars at PM K
yerr_lower = model_p_at_pmK − lo_at_pmK
yerr_upper = hi_at_pmK − model_p_at_pmK
yerr = np.vstack([yerr_lower, yerr_upper])
ax1.errorbar(pm_K, model_p_at_pmK, yerr=yerr, fmt=”o”, color=”#1f77b4”,
ecolor=”#1f77b4”, elinewidth=1.2, capsize=3, label=”Model @ PM K (±95%
CI)”, alpha=0.9, zorder=3)

# PM points
ax1.scatter(pm_K[~flagged], pm_p[~flagged], s=60, marker=”x”, color=”
#7f7f7f”, label=”Polymarket (not flagged)”, zorder=4)
ax1.scatter(pm_K[flagged], pm_p[flagged], s=70, marker=”D”, color=”
#d62728”, label=”Polymarket (flagged)”, zorder=5)

# S0 line
ax1.axvline(spot, ls=”:”, lw=1, color=”#444444”, label=f”S0 =
{spot:,.0f}”)
ax1.set_xlabel(”Threshold K”); ax1.set_ylabel(”Probability”)
ax1.set_title(f”Touch probability: Model vs Polymarket (T=
{T_years:.3f}y)”)
ax1.yaxis.set_major_formatter(FuncFormatter(lambda x, pos: f”{x:.0%}”))
ax1.grid(True, ls=”−−”, alpha=0.35)
ax1.legend(loc=”best”)

# Panel 2: spread with CI band and abs threshold
ci_margin_curve = np.minimum(sweep_df[”p_hit”] − sweep_df[”p_hit_lo95”],
sweep_df[”p_hit_hi95”] − sweep_df[”p_hit”]) ∗ 100.0
ax2.fill_between(sweep_df[”K”], −ci_margin_curve, ci_margin_curve,
color=”#1f77b4”, alpha=0.12, label=”± Model CI margin”)
ax2.axhline(y=+abs_thresh_pct, color=”#ff7f0e”, lw=1.5, ls=”−−”,
label=f”± Abs. threshold ({abs_thresh_pct:.1f}%)”)
ax2.axhline(y=−abs_thresh_pct, color=”#ff7f0e”, lw=1.5, ls=”−−”)
ax2.axhline(y=0.0, color=”#222222”, lw=1.0)
ax2.scatter(pm_K[~flagged], spreads_pct[~flagged], s=60, color=”
#7f7f7f”, marker=”o”, label=”Spread (PM − Model): not flagged”,
zorder=5)
ax2.scatter(pm_K[flagged], spreads_pct[flagged], s=70, color=”#2ca02c”,
marker=”o”, label=”Spread (PM − Model): flagged”, zorder=6)
ax2.set_xlabel(”Threshold K”); ax2.set_ylabel(”Spread (%)”)
ax2.set_title(”Spread vs K with gating bands”)
ax2.grid(True, ls=”−−”, alpha=0.35)
ax2.legend(loc=”best”)

# Footer caption
val_dt = rates_meta.get(”valuation_dt_iso”) if isinstance(rates_meta,
dict) else None
caption_left = f”Model valuation: {valuation_dt:%Y−%m−%d %H:%M} |
Expiry: {expiry_dt:%Y−%m−%d}”
caption_right = f”Generated: {datetime.now(timezone.utc):%Y−%m−%d %H:%M
UTC}”
fig.text(0.5, 0.01, f”{caption_left} | {caption_right}”, ha=”center”,
va=”bottom”, fontsize=9)

plt.show()
fig.savefig(PLOT_PNG, dpi=160)
print(f”\nSaved comparison figure to: {PLOT_PNG}”)

# Flagged opportunities table
flag_df = pd.DataFrame({
    “K”: pm_K,
    “Polymarket”: pm_p,
    “Model”: model_p_at_pmK,
    “Spread_pct”: spreads_pct,
    “CI_margin_pct”: ci_margin_pct_at_pmK,
    “Abs_threshold_pct”: np.full_like(spreads_pct, abs_thresh_pct),
    “Flagged”: flagged,
}).loc[np.argsort(pm_K)].reset_index(drop=True)

print(”\nFlagged opportunities (if any):”)
print(flag_df[flag_df[”Flagged”] == True])
```



```
Polymarket vs Model @ specified K:
K                 PM     Model    Spread    CI_mrg   AbsThr   Flag
50,000         0.020     0.007    +1.29%     0.04%    3.00%     no
70,000         0.080     0.067    +1.26%     0.11%    3.00%     no
80,000         0.150     0.173    -2.31%     0.17%    3.00%     no
90,000         0.360     0.406    -4.63%     0.21%    3.00%    YES
100,000        0.680     0.827   -14.66%     0.17%    3.00%    YES
120,000        0.420     0.342    +7.84%     0.21%    3.00%    YES
130,000        0.250     0.137   +11.29%     0.15%    3.00%    YES
140,000        0.130     0.058    +7.24%     0.10%    3.00%    YES
150,000        0.070     0.028    +4.20%     0.07%    3.00%    YES
170,000        0.030     0.009    +2.10%     0.04%    3.00%     no
200,000        0.010     0.003    +0.74%     0.02%    3.00%     no
250,000        0.020     0.001    +1.94%     0.01%    3.00%     no
```



Model valuation: 2025-11-12 06:37 | Expiry: 2026-01-01 | Generated: 2025-11-12 08:36 UTC

Conclusion

I hope you enjoyed this article. I'm excited to announce that I've expanded the team and we can now create even more high quality articles like this!

Discord: https://discord.gg/X7TsxKNbXg

Subscribe to VertoxQuant

Hundreds of paid subscribers

Quantitative Research

Upgrade to paid

1 Like · 1 Restack

1 · 1 · Share · ···

Discussion about this post

Comments · Restacks

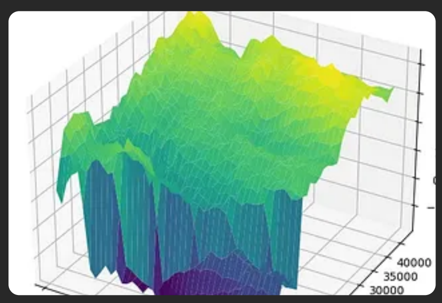
Write a comment...

Top · Latest · Discussions

Quantitative Finance - How to Start

A full roadmap for how to get started in quant finance.

DEC 10, 2024 · VERTOX



A Full Guide to Risk Management

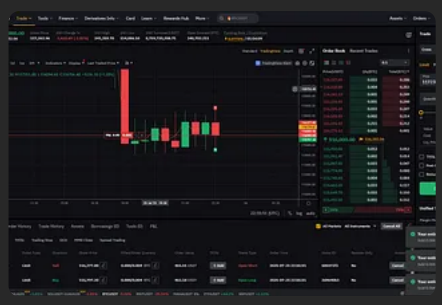
Everything you need to know about risk management

JUN 14 · VERTOX AND MALIK

How to Start a Market Making Business from Scratch

Building a market making engine and all the other components that go into it

JUL 26 · VERTOX AND MALIK



See all >

Ready for more?

Upgrade to paid

© 2025 Vertox · Privacy · Terms · Collection notice

Start your Substack · Get the app

Substack is the home for great culture